



UNIVERSIDADE
LUSÓFONA

Aplicação RentScope

Trabalho Final de Curso

Licenciatura em Engenharia Informática

Projeto II

Relatório

2º Semestre

Júlia Santos, 22301020

Orientador: Hugo Fernando Azevedo Barbosa

Departamento de Engenharia Informática e Sistemas de Informação
Universidade Lusófona, Centro Universitário do Porto (CUP)

MAIO 2026

Direitos de cópia

Aplicação RentScope, Copyright de Júlia Santos, Universidade Lusófona.

A Faculdade de Ciências Naturais, Engenharia e Tecnologias (FCNET) e a Universidade Lusófona têm o direito, perpétuo e sem limites geográficos, de arquivar e publicar esta dissertação/monografia através de exemplares impressos reproduzidos em papel ou de forma digital, ou por qualquer outro meio conhecido ou que venha a ser inventado, e de a divulgar através de repositórios científicos e de admitir a sua cópia e distribuição com objetivos educacionais ou de investigação, não comerciais, desde que seja dado crédito ao autor e editor.

Agradecimentos

A mim própria, pela resiliência e dedicação ao longo de todo este percurso acadêmico por não ter desistido nos momentos mais difíceis e por ter conseguido concluir este projeto do início ao fim.

Ao Maurício, por estar presente em todas as fases deste processo.

Ao meu orientador e a todos os professores e colegas que, de forma direta ou indireta, contribuíram para a minha formação e para o desenvolvimento deste trabalho.

À minha família, pelo suporte incondicional, mesmo à distância

Resumo

O projeto consiste no desenvolvimento de uma aplicação móvel que apresenta visualmente os preços médios de aluguel por cidade, utilizando dados provenientes de bases oficiais. A aplicação exibirá um mapa coroplético em que a intensidade das cores representa a variação dos preços, permitindo a identificação rápida de regiões mais acessíveis ou onerosas. Vão ser implementados filtros que permitem ao utilizador cruzar informações de preço com outros fatores, como criminalidade, acesso a transportes públicos e disponibilidade de serviços locais. O sistema vai ser desenvolvido priorizando a precisão e a confiabilidade das fontes de dados. Os resultados esperados deverão demonstrar se a visualização geográfica dos preços auxilia o processo de tomada de decisão de pessoas que procuram moradia em locais desconhecidos, mediante uma análise comparativa eficiente entre diferentes regiões.

Palavras-chave: Aplicação móvel; Aluguel; Mapa coroplético; Visualização de dados

Abstract

The project consists of developing a mobile application that visually presents the average rental prices by city, using data from official sources. The application will display a choropleth map in which the color intensity represents price variations, allowing users to quickly identify more affordable or expensive regions. Filters will be implemented to enable users to cross-reference price information with other factors, such as crime rates, access to public transportation, and availability of local services. The system will be developed with a focus on data accuracy and reliability. The expected results should demonstrate whether the geographical visualization of prices assists the decision-making process for people seeking housing in unfamiliar locations, through an efficient comparative analysis between different regions.

Keywords: Mobile application; Rent; Choropleth map; Data visualization

Sumário

Resumo	4
Abstract	5
Lista de Figuras.....	8
Lista de Tabelas.....	9
Lista de Siglas/Significados.....	10
1 Introdução.....	11
1.1 Enquadramento.....	11
1.2 Motivação e Identificação do Problema.....	11
1.3 Objetivos	11
1.4 Estrutura do Documento.....	12
2 Pertinência e Viabilidade.....	1
2.1 Pertinência	1
2.2 Viabilidade.....	1
2.3 Análise Comparativa com soluções existentes.....	5
2.3.1 Soluções existentes.....	5
2.3.2 Análise de benchmarking.....	7
2.4 Proposta de inovação e mais-valias	7
2.5 Identificação de oportunidade de negócio.....	8
3 Especificação e Modelação	10
3.1 Análise de requisitos	10
3.1.1 Requisitos funcionais	10
3.1.2 Requisitos não-funcionais	11
3.1.3 Casos de Uso	13
3.2 Modelação.....	14
3.2.1 Entidade-Relação.....	14
3.2.2 Diagrama de Classes	14
3.2.3 Diagrama de Componentes	15
3.3 Protótipos de Interface	16
3.3.1 Protótipos de Interface de Baixa Fidelidade	16
3.3.2 Protótipos de Interface de Alta-Fidelidade	20
4 Solução Desenvolvida	26

4.1	Introdução	26
4.2	Arquitetura	26
4.3	Tecnologias e Ferramentas Utilizadas	28
4.4	Ambientes de Teste e de Produção.....	29
4.5	Abrangência	31
4.6	Componentes	32
4.6.1	Componente 1 – Dados, ETL e Base de Dados.....	32
4.6.2	Componente 2 – API e Integração com a Base de Dados.....	33
4.6.3	Componente 3 - Aplicação Android e Consumo API.....	33
4.6.4	Componente 4 – Visualização Geográfica com Google Maps e GeoJSON	34
4.6.5	Componente 5 – Mascote Interativo	34
4.6.6	Componente 6 – Assistente Inteligente	35
4.7	Interfaces.....	36
5	Testes e Validação	42
5.1	Abordagem e Justificação dos Testes	42
5.2	Ambientes de Teste	42
5.3	Testes Funcionais.....	43
5.4	Testes de Integração.....	43
5.5	Testes de Validação Operacional.....	43
5.6	Testes de Usabilidade	44
5.7	Análise de Riscos e Impacto.....	44
6	Método e Planeamento	44
6.1	Metodologia de Desenvolvimento.....	44
6.2	Plano de Trabalho e Cronograma	45
6.3	Análise Crítica do Cumprimento do Plano	45
7	Resultados dos Testes.....	46
7.1	Descrição dos Resultados.....	46
7.1.2	Outputs e Outcomes	47
7.1.3	Validação por Terceiros.....	47
7.2	Cumprimentos de Requisitos.....	47
7.2.1	Requisitos Funcionais.....	47
7.2.2	Requisitos não-funcionais	48
7.3	Funcionalidades Adicionais e Melhorias Introduzidas.....	48
8	Conclusão.....	49

Bibliografia	50
Anexo A — Guião de Testes e Resultados.....	51

Lista de Figuras

Figura 1- Gráfico de pizza demonstrando as repostas referentes a pergunta 1 do inquérito	2
Figura 2 - Gráfico de pizza demonstrando as repostas referentes a pergunta 2 do inquérito	2
Figura 3 – Gráfico de pizza demonstrando as repostas referentes a pergunta 3 do inquérito.....	3
Figura 4– Gráfico de pizza demonstrando as repostas referentes a pergunta 4 do inquérito.....	3
Figura 5– Gráfico de barras demonstrando as repostas referentes a pergunta 5 do inquérito	4
Figura 6 – Gráfico de pizza demonstrando as repostas referentes a pergunta 6 do inquérito.....	4
Figura 7– Página inicial do site Idealista	5
Figura 8– Visualização geral da aplicação Idealista	5
Figura 9 – Página inicial do site RentHub.....	6
Figura 10 – Página inicial da ferramenta Affordable Housing Atlas	6
Figura 11 – Análise SWOT	9
Figura 12 – Casos de uso.....	13
Figura 13 – Diagrama Entidade-Relação.....	14
Figura 14 – Diagrama de Classes	14
Figura 15 – Diagrama de Componentes	15
Figura 16 – Mockup 1	16
Figura 17 – Mockup 2	17
Figura 18 – Mockup 3	18
Figura 19 – Mockup 4	19
Figura 20 – Mockup 5	20
Figura 21 – Mockup 6	21
Figura 22 – Mockup 7	22
Figura 23 – Mockup 8	23
Figura 24 – Mockup 9	24
Figura 25 – Mockup 10	25
Figura 26 - Arquitetura do Software	27
Figura 27 - QR code para baixar a aplicação	30
Figura 28 - Home Screen.....	37
Figura 29 - IA Screen	38
Figura 30 - Favorites Screen.....	39
Figura 31 - Search Country Screen.....	40
Figura 32 - Map Screen	41
Figura 33 - Diagrama de Gantt.....	45

Lista de Tabelas

Tabela 1 – Análise de benchmarking	7
Tabela 2 – Página de Login	10
Tabela 3 – Mapa Coroplético Interativo	10
Tabela 4 – Filtros Personalizados.....	10
Tabela 5 – Comparação de Localidades.....	10
Tabela 6 – Análise Temporal.....	10
Tabela 7 – Perfil do Utilizador.....	11
Tabela 8 – Mascote Interativo	11
Tabela 9 – Gestão da Base de Dados	11
Tabela 10 – Interface de Navegação.....	11
Tabela 11 – Usabilidade.....	11
Tabela 12 – Confiabilidade dos Dados.....	11
Tabela 13 - Segurança.....	12
Tabela 14 – Desempenho	12
Tabela 15 – Disponibilidade.....	12
Tabela 16 – Escalabilidade	12
Tabela 17 - Cumprimento dos Requisitos Funcionais.....	48
Tabela 18 - Cumprimento dos Requisitos Não-Funcionais.....	48
Tabela 19 - Tarefas e Objetivos dos Testes.....	51
Tabela 20 - Resultados dos Testes.....	51

Lista de Siglas/Significados

API – Application Programming Interface, conjunto de definições e protocolos que permite a comunicação entre diferentes componentes de software.

ASGI – Asynchronous Server Gateway Interface, especificação que define como servidores web comunicam com aplicações Python assíncronas.

CAOP – Carta Administrativa Oficial de Portugal, base de dados oficial dos limites administrativos do território português.

CSV – Comma-Separated Values, formato de ficheiro tabular em texto simples utilizado para a troca de dados.

ETL – Extract, Transform, Load, processo de extração, transformação e carregamento de dados entre sistemas.

GeoJSON – Formato de dados geoespaciais baseado em JSON para representar entidades geográficas.

INE – Instituto Nacional de Estatística, organismo oficial responsável pela produção e divulgação de informação estatística em Portugal.

JSON – JavaScript Object Notation, formato leve de troca de dados estruturados.

JWT – JSON Web Token, padrão para representação compacta e segura de declarações entre duas partes, utilizado em mecanismos de autenticação.

MFA – Multi-Factor Authentication, mecanismo que exige múltiplos fatores de identificação para autenticar um utilizador.

NUTS – Nomenclatura das Unidades Territoriais para fins Estatísticos, sistema hierárquico de divisão regional usado na União Europeia.

REST – Representational State Transfer, estilo arquitetural para a construção de serviços web.

SDK – Software Development Kit, conjunto de ferramentas de desenvolvimento de software para uma plataforma específica.

SMTP – Simple Mail Transfer Protocol, protocolo padrão para o envio de correio eletrónico.

SQL – Structured Query Language, linguagem padrão para consulta e manipulação de bases de dados relacionais.

Coroplético - Mapa temático que utiliza diferentes cores, tons, padrões ou sombreamentos para representar a distribuição quantitativa de um determinado fenómeno numa área geográfica.

1 Introdução

1.1 Enquadramento

Ao se mudar para uma região a qual não se tem familiaridade seja para grandes centros urbanos ou regiões menos povoadas, encontrar um local para morar geralmente é um processo confuso. O mercado imobiliário apresenta variações significativas de preços de aluguel entre diferentes localidades consoante a variáveis como número de escolas, taxa de criminalidade, número de hospitais na região etc. Todos esses parâmetros dificultam a comparação e a tomada de decisão para uma pessoa que não é natural da região. Para tornar esse processo mais acessível, a aplicação consiste em utilizar-se de um mapa coroplético para representar visualmente os preços de aluguéis em diferentes lugares do mundo. Assim, o utilizador poderá rapidamente identificar zonas mais caras ou mais acessíveis, filtrando de acordo com suas preferências, de forma intuitiva.

1.2 Motivação e Identificação do Problema

A escolha deste tema surgiu da observação de uma dificuldade recorrente enfrentada por muitas pessoas de fora da comunidade que se deseja integrar ao procurar um imóvel para arrendar: a falta de acesso informações integradas que permitam comparar diferentes regiões de forma clara e objetiva. Embora existam plataformas de arrendamento, estas geralmente limitam-se a apresentar listagens de imóveis, sem fornecer uma análise espacial que ajude o utilizador a compreender quais áreas melhor se adequam às suas preferências e necessidades. Neste contexto, a aplicação visa criar uma solução tecnológica que combine dados geográficos e estatísticos para que então um mapa coroplético interativo permita o utilizador visualizar e cruzar variáveis relevantes, como preço médio, acessibilidade, segurança e infraestrutura, a fim de identificar a região mais adequada ao seu perfil. Eu lidei pessoalmente com esse problema quando vim do Brasil para o Porto para fazer a presente faculdade e precisei arrendar um imóvel, nenhum software que eu tinha conhecimento levava em consideração o meu perfil e as minhas preferências, dessa forma acredito que consigo desenvolver um olhar mais atento a realidade de pessoas que são de fora da região onde pretende-se morar.

1.3 Objetivos

Desenvolver uma aplicação móvel interativa que permita visualizar, em formato de mapa coroplético, os preços de aluguéis de imóveis – por região - em diferentes localidades do mundo. Objetivos específicos:

- Representar de forma visual e simplificada as diferenças de preço de aluguel entre diversas localidades;
- Permitir que o utilizador identifique rapidamente zonas mais caras ou acessíveis de acordo com suas preferências pessoais;
- Disponibilizar filtros que levem em conta variáveis relevantes;
- Recolher e organizar dados de fontes confiáveis de bases de dados oficiais;
- Oferecer uma interface amigável, acessível e interativa;
- Facilitar a tomada de decisão em processos de mudança.

1.4 Estrutura do Documento

O presente relatório está organizado da seguinte forma:

- Na Secção 1 é apresentado o enquadramento do projeto, a motivação, o problema identificado e os objetivos definidos.
- Na Secção 2 é analisada a pertinência e viabilidade da solução proposta, incluindo a comparação com soluções existentes, a proposta de inovação e a identificação de oportunidades de negócio.
- Na Secção 3 é apresentada a especificação do sistema, incluindo a análise de requisitos, a modelação e os protótipos de interface.
- Na Secção 4 é descrita a solução desenvolvida, incluindo a arquitetura, as tecnologias utilizadas, os ambientes de teste e produção, os principais componentes e as interfaces finais da aplicação.
- Na Secção 5 é apresentada a estratégia de testes e validação adotada para verificar o correto funcionamento do sistema.
- Na Secção 6 é descrita a metodologia de desenvolvimento utilizada, bem como o planeamento e o cronograma do projeto.
- Na Secção 7 são apresentados os resultados obtidos nos testes, incluindo a análise do cumprimento dos requisitos e a validação da solução.
- Na Secção 8 são apresentadas as conclusões finais do trabalho, bem como possíveis melhorias e desenvolvimentos futuros.
- Na Bibliografia encontram-se listadas as referências utilizadas ao longo do relatório.
- No Anexo A é disponibilizado o guião de testes e os respetivos resultados.

O quadro abaixo é indicativo de entregáveis (*deliverables*/relatórios) para cada momento de avaliação, assumindo abordagem sequencial do desenvolvimento do trabalho de final de curso (TFC), incorporando as unidades curriculares de **Projeto I** (1º semestre) e **Projeto II** (2º semestre). Podendo ser adotadas outras abordagens metodológicas (e.g.: metodologias ágeis), serão aceites outras organizações de entregas no âmbito do desenvolvimento das UCs Projeto I e Projeto II. Deve-se, no entanto, observar duas condições: (i) as entregas deverão manter os conteúdos indicados no quadro, por forma a permitir ao júri (professores) avaliar a pertinência do tema e a taxa de esforço esperada; (ii) a organização de conteúdos em cada entrega deve ter atenção aos critérios de avaliação de modo a garantir a uniformidade da avaliação das UCs.

Quadro de conteúdos – desenvolvimento (Projeto I e Projeto II)

Avaliação	Tipo	1. Introdução	2. Pertinência e Viabilidade	3. Especificação e Modelação	4. Solução (arquitetura) desenvolvida ¹	5. Testes e validação	6. Método e planeamento	7. Resultados	8. Conclusão
Entrega Intercalar Projeto I (Nov 2025)	Qualitativa	Incluir	Incluir	Incluir	Opcional	N/A	Incluir	Opcional	Opcional
Entrega relatório final Projeto I (Jan 2026)	Quantitativa	Incluir	Incluir	Incluir	Opcional	N/A	Incluir	Opcional	Incluir (no âmbito de Projeto I)
Entrega Intercalar Projeto II (Abril 2026)	Qualitativa	Incluir	Incluir	Incluir	Incluir	Incluir	Incluir	Opcional	Opcional
Entrega relatório final² Projeto II (Maio 2026)	Quantitativa	Final	Final	Final	Final	Final	Final ³	Final	Final

¹ Sempre que aplicável, inclui código funcional, a disponibilizar em repositório *Git*

² O relatório final deverá incluir todos os conteúdos desenvolvidos ao longo do TFC (Projeto I e Projeto II)

³ Para a entrega final sugere-se a inclusão, em anexo, de todo os planos de trabalho anteriores com apreciação de progresso e avaliação das dificuldades encontradas na elaboração e cumprimento do planeamento

2 Pertinência e Viabilidade

2.1 Pertinência

O trabalho em desenvolvimento apresenta um impacto positivo na resolução do problema identificado anteriormente, relacionado com a dificuldade de acesso a informações centralizadas, comparáveis e visuais sobre o mercado de arrendamento com foco no utilizador final. A aplicação proposta permite que utilizadores explorem, através de um mapa coroplético interativo, os preços médios de arrendamento por região, cruzando esses dados com critérios personalizados, como orçamento disponível e infraestrutura no geral. Esta abordagem facilita uma tomada de decisão mais eficiente, promovendo uma melhora na procura de habitação. O projeto também contribui socialmente, ao apoiar estudantes, famílias e profissionais que se deslocam para diferentes regiões e necessitam de compreender rapidamente o custo de vida, o contexto e as oportunidades de arrendamento em cada localidade.

2.2 Viabilidade

Em termos económicos, o projeto demonstra potencial de continuidade e escalabilidade. A solução proposta pode ser transformada num produto digital viável, oferecendo valor tanto para utilizadores individuais, que procuram tomar decisões informadas sobre onde alugar, quanto para entidades do setor imobiliário e instituições públicas interessadas em compreender dinâmicas regionais de arrendamento. Sob o ponto de vista social, o projeto contribui positivamente para o mercado habitacional, promovendo a transparência e o acesso à informação sobre preços e qualidade de vida nas diferentes regiões. Tal impacto reforça o potencial de aceitação pública e o interesse por parte de entidades externas em apoiar ou utilizar a aplicação. A capacidade de integrar e cruzar dados relevantes de forma intuitiva indica que a solução não se esgota enquanto projeto académico, podendo evoluir para um produto real com aplicabilidade prática e sustentação no mercado. Com base nos resultados de um inquérito realizado através do “Google Forms”, observa-se que a aplicação apresenta uma proposta de valor para utilizadores reais.

Link do formulário: <https://forms.gle/YvtdbEtaW2yKQUGf9>

Qual a sua faixa etária?

 Copiar gráfico

46 respostas

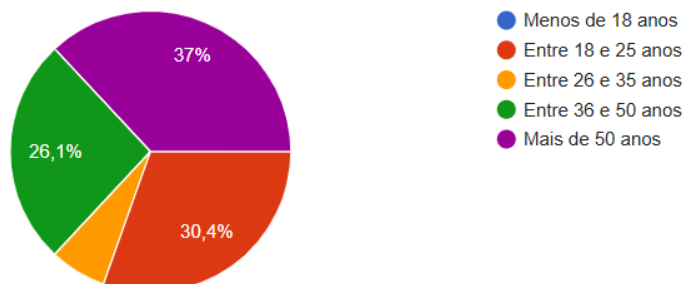


Figura 1- Gráfico de pizza demonstrando as repostas referentes a pergunta 1 do inquérito

Os respondentes apresentam uma distribuição etária variada, com maior predominância de pessoas com mais de 50 anos e entre 18 e 25 anos, indicando a participação tanto de jovens quanto de adultos mais experientes.

Procurou imóvel para arrendar (alugar) nos últimos 5 anos?

 Copiar gráfico

46 respostas

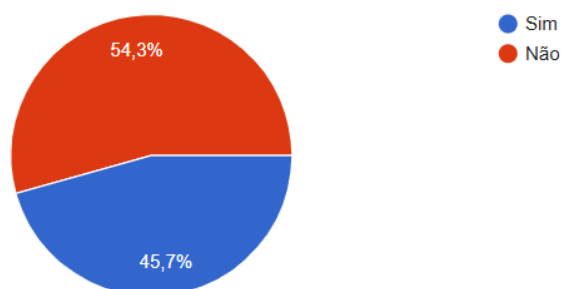


Figura 2 - Gráfico de pizza demonstrando as repostas referentes a pergunta 2 do inquérito

A maioria dos participantes não procurou imóvel para arrendar nos últimos cinco anos, embora uma parcela significativa tenha passado por essa experiência recentemente.

Pretende arrendar (alugar) um imóvel nos próximos 5 anos?

 Copiar gráfico

46 respostas

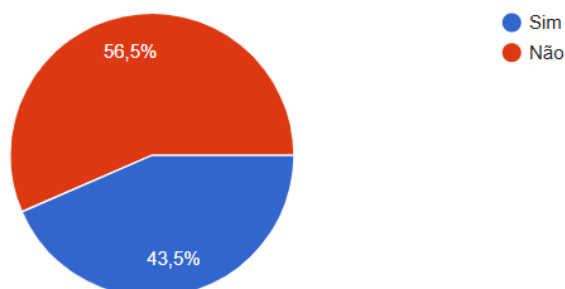


Figura 3 – Gráfico de pizza demonstrando as repostas referentes a pergunta 3 do inquérito

Grande parte dos respondentes não pretende arrendar um imóvel nos próximos cinco anos, porém uma percentagem relevante demonstra essa intenção.

Considera fácil encontrar informações fiáveis sobre preços de arrendamento (aluguel) com filtros como transportes públicos próximos e segurança na região?

 Copiar gráfico

46 respostas

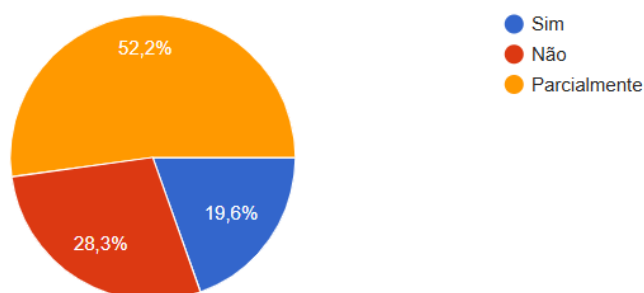


Figura 4– Gráfico de pizza demonstrando as repostas referentes a pergunta 4 do inquérito

A maioria dos participantes considera apenas parcialmente fácil encontrar informações fiáveis sobre preços, transportes públicos e segurança, evidenciando dificuldades no acesso a dados completos e confiáveis.

O que achas de um aplicativo que apresente um mapa interativo com preços médios de arrendamento (aluguel), filtrando preferências pessoais, por região?

 Copiar gráfico

46 respostas

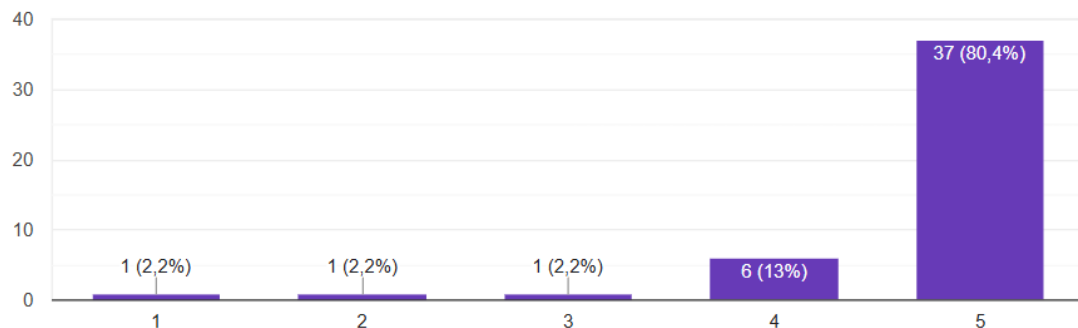


Figura 5– Gráfico de barras demonstrando as repostas referentes a pergunta 5 do inquérito

Os respondentes demonstraram uma avaliação muito positiva da ideia, com a maioria atribuindo a pontuação máxima, indicando forte interesse e aceitação da solução proposta.

Utilizaria esta ferramenta se estivesse disponível gratuitamente online?

 Copiar gráfico

46 respostas

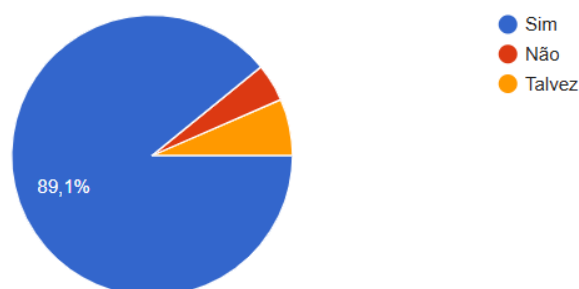


Figura 6 – Gráfico de pizza demonstrando as repostas referentes a pergunta 6 do inquérito

A maioria absoluta dos participantes afirmou que utilizaria a ferramenta caso estivesse disponível gratuitamente online, mostrando elevado potencial de adesão.

2.3 Análise Comparativa com soluções existentes

2.3.1 Soluções existentes

Idealista

Oferece um mapa que mostra quanto custa arrendar casa em cada município de Portugal. Permite visualizar diferenças geográficas de forma simples: interior x litoral, municípios mais caros x mais baratos. Embora visualize os preços, não oferece filtros personalizados extensivos ou cruzamentos detalhados das preferências do utilizador com os dados. As duas soluções têm o componente de mapa interativo.



Figura 7—Página inicial do site Idealista

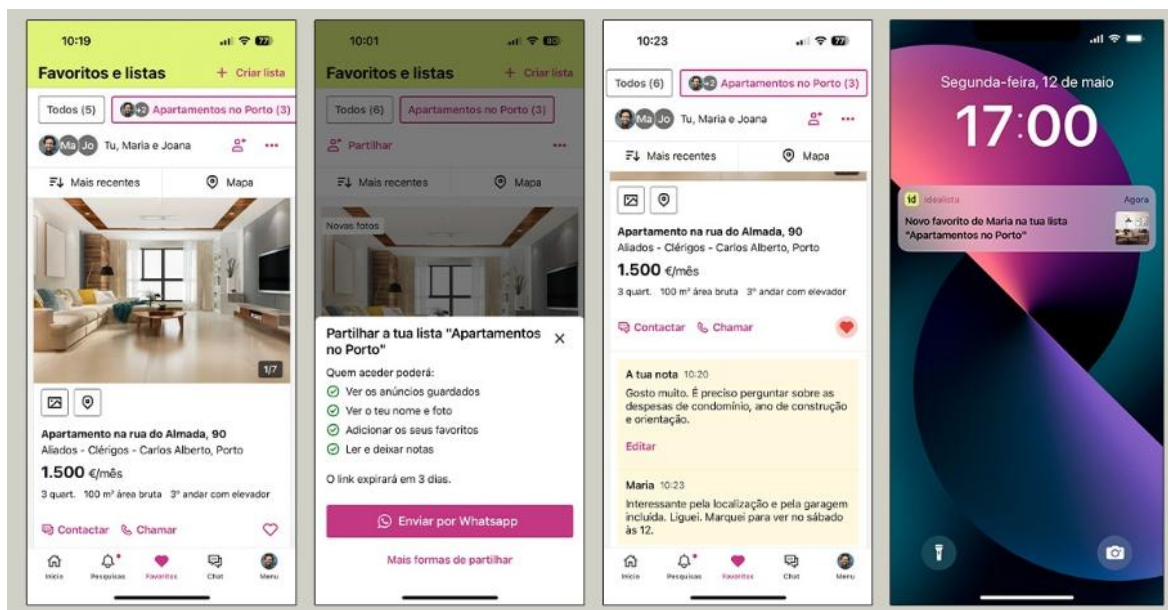


Figura 8—Visualização geral da aplicação Idealista

RentHub

Fornece dashboards de dados de arrendamento nos EUA, incluindo mapas de calor, médias de rendas por código-postal etc. Oferece filtros de para examinar diferentes configurações de zonas geográficas. Essa solução relaciona-se com o meu projeto mostrando como o uso de visualização geográfica + dados agregados de arrendamento já existe num mercado global (EUA) mas com foco mais orientado para investidores e profissionais de mercado e menos para o utilizador final com filtros personalizados.



Rental Housing Data and Insights to Help You Win Your Markets

Our dashboards give you an instant snapshot of everything you need to keep tabs on your markets. Understand key analytics like rents, amenity premiums, and other metrics!

Try it for Free

Watch Demo

Popular cities ● LIVE TICKER

NEW YORK

CHICAGO

LOS ANGELES

HOME

ABOUT

PRICING

CONTACT

LOGIN

SIGN UP FOR FREE

Rent Trends



Median Price/Sqft

1.81%
\$3.25

Stainless



Figura 9 – Página inicial do site RentHub

Affordable Housing Atlas

Ferramenta global que permite analisar acessibilidade habitacional, cruzando múltiplos indicadores (aluguéis, renda, crescimento urbano) em várias regiões metropolitanas no mundo. Demonstra que o problema da acessibilidade ao arrendamento é global e que há interesse em visualizações comparativas de várias regiões.



Figura 10 – Página inicial da ferramenta Affordable Housing Atlas

2.3.2 Análise de benchmarking

Com o objetivo de avaliar a solução a ser desenvolvida em relação as já existentes no mercado, foi realizada uma análise comparativa entre o presente projeto e Idealista, Renthub e Affordable Housing Atlas. O Idealista é uma das principais plataformas de pesquisa de imóveis em Portugal e na Espanha, focada na divulgação de anúncios e contatos entre proprietários, imobiliárias e potenciais arrendatários. Já o Renthub, de origem italiana, oferece análise de preços de arrendamento com base em dados de mercado, funcionando como um observatório de valores imobiliários. Por fim, o Affordable Housing Atlas, desenvolvido em contexto académico, apresenta uma visão geoespacial sobre a acessibilidade habitacional, cruzando dados económicos e sociais. A tabela seguinte resume as principais características comparadas entre as quatro soluções:

	RentScope	RentHub	Idealista	Affordable Housing Atlas
Visualização geográfica em mapa	x	x	x	x
Análise estatística de preços médios de arrendamento	x	x	x	x
Filtragem por preferências do utilizador	x		x	
Foco em acessibilidade habitacional e contexto socioeconómico	x			x
Aplicação interativa e exploratória	x	x	x	x
Comparação entre regiões	x	x		
Pesquisa global	x			x

Tabela 1 – Análise de benchmarking

2.4 Proposta de inovação e mais-valias

O principal diferencial da RentScope em relação às soluções existentes é uma abordagem com maior ênfase no utilizador final que está a arrendar, integrando dados de preferência pessoal com base no contexto e na infraestrutura que cada um está buscando visando ajudar pessoas que não tem noção da região onde vão morar, além de cobrir vários países ou regiões comparativas. Como uma mais-valia também é possível referir à integração de inteligência artificial.

2.5 Identificação de oportunidade de negócio

O mercado imobiliário global enfrenta atualmente um desafio comum: a falta de transparência e centralização dos dados de arrendamento, o que dificulta decisões informadas por parte dos utilizadores. Apesar da existência de plataformas conhecidas, como Idealista ou RentHub, a maioria delas centra-se na divulgação de anúncios individuais e não na análise integrada de dados de mercado. A oportunidade de negócio surge, portanto, na lacuna entre os portais imobiliários tradicionais e as ferramentas de análise de dados profissionais:

- Os primeiros são acessíveis, mas oferecem pouca inteligência analítica.
- Os segundos são poderosos devido aos dados que possuem, mas inacessíveis ao público geral (por custos ou complexidade).

O projeto posiciona-se como uma solução intermediária, oferecendo informação analítica, visual e personalizada a um público mais vasto.

Potenciais segmentos de mercado

- Estudantes e jovens profissionais – grupo que frequentemente muda de cidade e precisa identificar regiões com rendas acessíveis e boa infraestrutura.
- Famílias em processo de mudança – procuram equilibrar preço, qualidade de vida e proximidade de serviços.
- Nômades digitais – interessados em comparar custo de arrendamento entre cidades ou países.
- Agentes imobiliários e gestores urbanos – podem utilizar os dados da aplicação para compreender tendências e apoiar decisões de investimento ou políticas habitacionais.

Valor agregado

- Acesso centralizado a dados dispersos (INE, Eurostat, portais locais, dados públicos internacionais).
- Visualização intuitiva via mapa coroplético, permitindo comparar regiões rapidamente.
- Filtros personalizados com base em orçamento, preferências, tipologia e outros critérios.
- Aplicabilidade internacional, podendo funcionar para diferentes países e bases de dados.
- Potencial de monetização através de:
 - modelo premium (funcionalidades básicas gratuitas e avançadas pagas);
 - subscrições para agências imobiliárias e municípios;
 - parcerias com plataformas de arrendamento;
 - anúncios dentro da aplicação.

SWOT Analysis Diagram

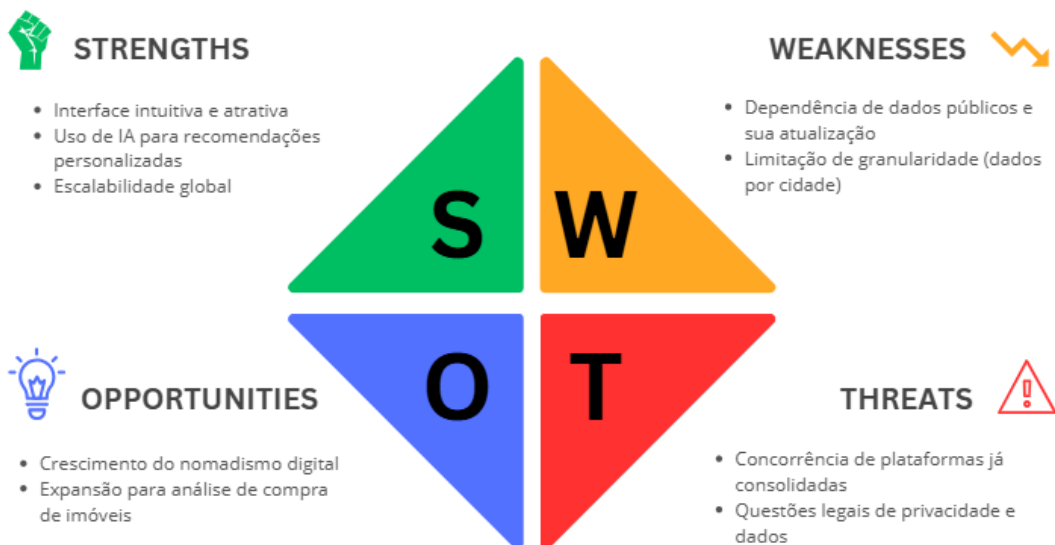


Figura 11 – Análise SWOT

3 Especificação e Modelação

3.1 Análise de requisitos

3.1.1 Requisitos funcionais

REQ.001	Página de Login
Prioridade	Essencial
Descrição	Permitir que o usuário crie um perfil utilizando seu e-mail na aplicação.

Tabela 2 – Página de Login

REQ.002	Mapa Coroplético Interativo
Prioridade	Essencial
Descrição	Permitir a visualização de regiões com preços de aluguel mais altos ou mais baixos, com possibilidade de ajustar o nível de detalhe (global, nacional ou regional).

Tabela 3 – Mapa Coroplético Interativo

REQ.003	Filtros Personalizados
Prioridade	Essencial
Descrição	Permitir que o utilizador pesquise por localização, faixa de preço e outras variáveis como transporte, escolas, segurança, áreas verdes e pontos de interesse.

Tabela 4 – Filtros Personalizados

REQ.004	Comparação de Localidades
Prioridade	Condicional
Descrição	Possibilitar a comparação lado a lado entre diferentes áreas quanto a preços médios e fatores relevantes.

Tabela 5 – Comparação de Localidades

REQ.005	Análise Temporal
Prioridade	Condicional
Descrição	Exibir o histórico de preços de aluguel e tendências de valorização ou desvalorização por região.

Tabela 6 – Análise Temporal

REQ.006	Perfil do Utilizador
Prioridade	Essencial
Descrição	Permitir que o utilizador salve áreas favoritas e configure alertas personalizados para variações de preço.

Tabela 7 – Perfil do Utilizador

REQ.007	Mascote Interativo
Prioridade	Condicional
Descrição	Incluir um guia virtual que auxilie na navegação e apresente dicas e explicações de forma acessível.

Tabela 8 – Mascote Interativo

REQ.008	Gestão da Base de Dados
Prioridade	Essencial
Descrição	Criar, armazenar e atualizar informações sobre preços de aluguel, variáveis complementares e perfis de utilizadores numa base de dados estruturada.

Tabela 9 – Gestão da Base de Dados

REQ.009	Interface de Navegação
Prioridade	Essencial
Descrição	Fornecer uma interface interativa que permita ao utilizador explorar os mapas e aplicar filtros facilmente.

Tabela 10 – Interface de Navegação

3.1.2 Requisitos não-funcionais

REQNF.001	Usabilidade
Prioridade	Essencial
Descrição	A interface deve ser intuitiva, amigável e acessível para utilizadores com diferentes níveis de experiência.

Tabela 11 – Usabilidade

REQNF.002	Confiabilidade dos Dados
Prioridade	Essencial
Descrição	Os dados exibidos devem ser provenientes de fontes oficiais e atualizadas.

Tabela 12 – Confiabilidade dos Dados

REQNF.003	Segurança
Prioridade	Essencial
Descrição	As informações do perfil do utilizador e suas preferências devem ser armazenadas de forma segura, utilizando a autenticação multifatorial.

Tabela 13 - Segurança

REQNF.004	Desempenho
Prioridade	Essencial
Descrição	O sistema deve apresentar respostas rápidas ao aplicar filtros e carregar mapas.

Tabela 14 – Desempenho

REQNF.005	Disponibilidade
Prioridade	Essencial
Descrição	O sistema deve estar disponível para uso contínuo, com mínima ou nenhuma interrupção dos serviços.

Tabela 15 – Disponibilidade

REQNF.006	Escalabilidade
Prioridade	Essencial
Descrição	O sistema deve ser capaz de suportar a adição de novas fontes de dados e variáveis de filtragem no futuro.

Tabela 16 – Escalabilidade

3.1.3 Casos de Uso

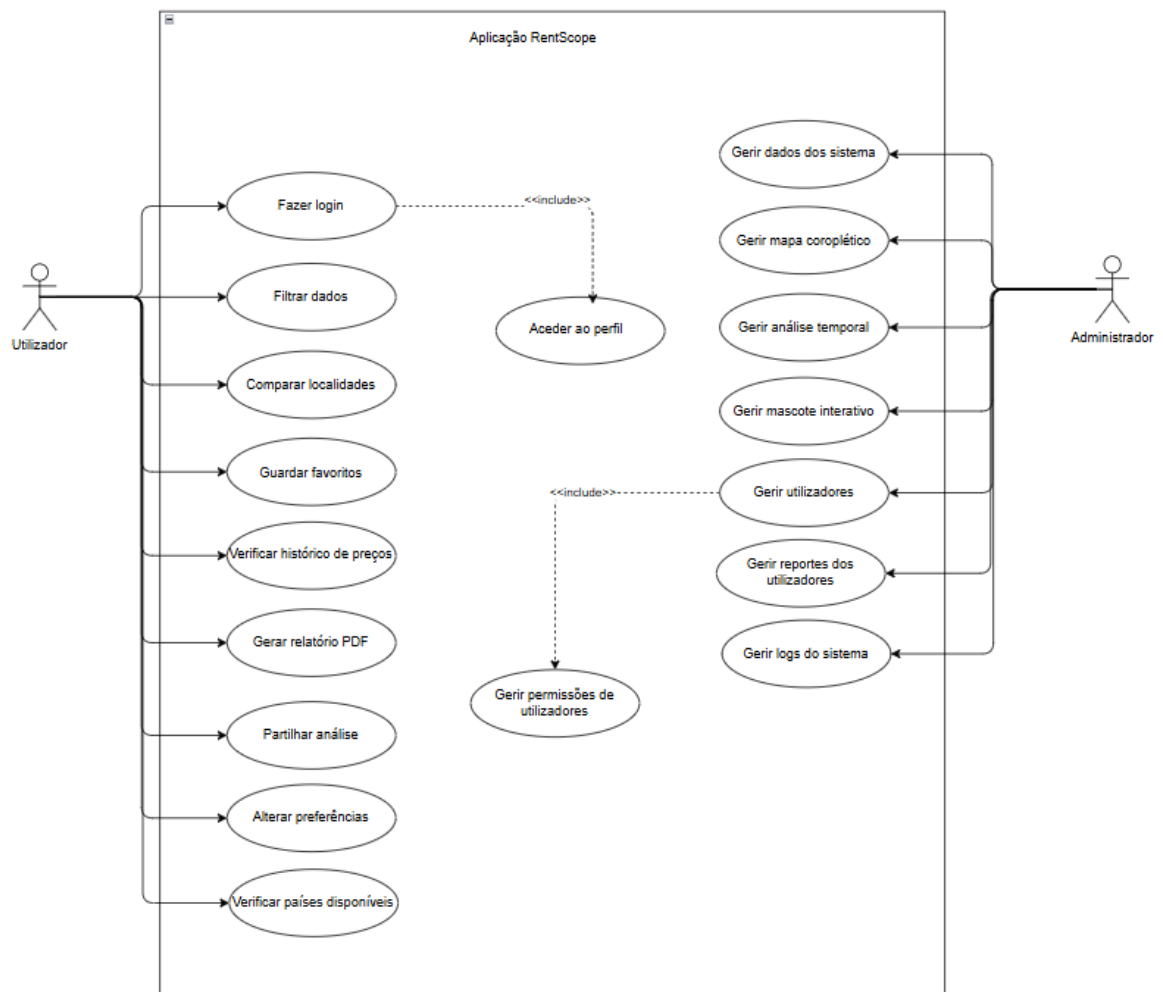


Figura 12 – Casos de uso

3.2.3 Diagrama de Componentes

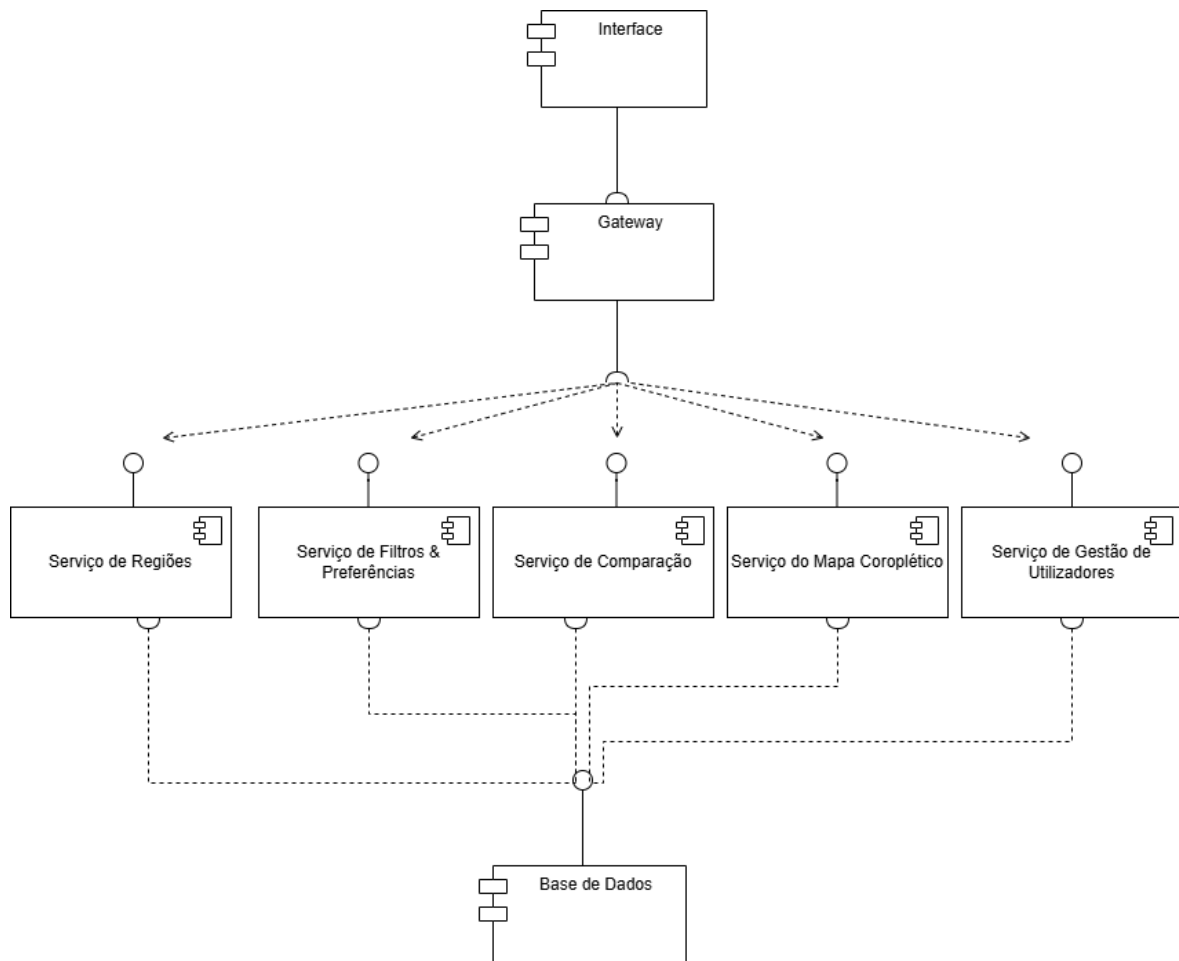


Figura 15 – Diagrama de Componentes

3.3 Protótipos de Interface

3.3.1 Protótipos de Interface de Baixa Fidelidade

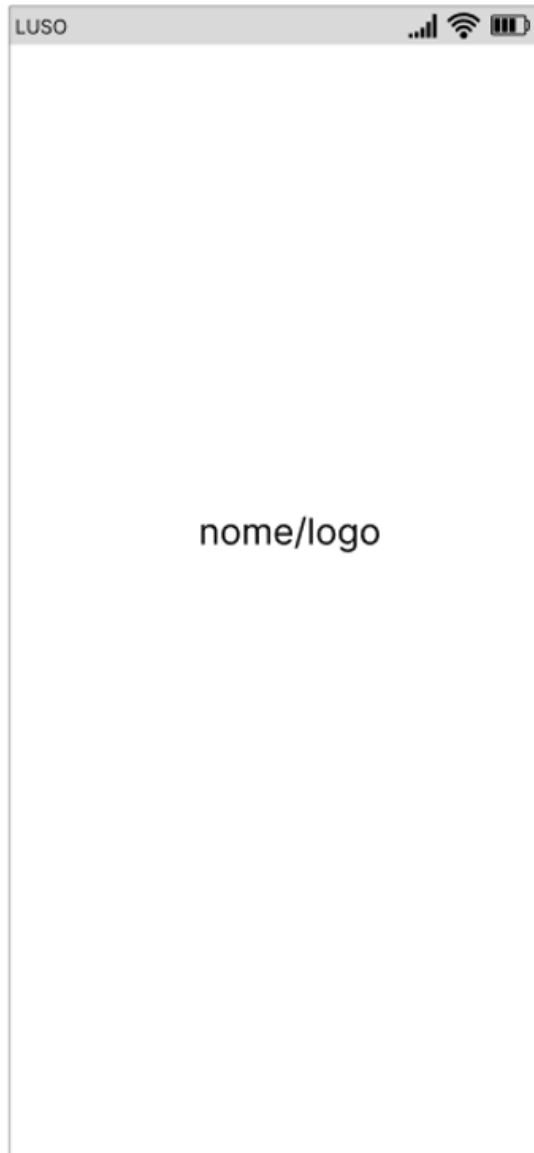


Figura 16 – Mockup 1



Figura 17 – Mockup 2



Figura 18 – Mockup 3

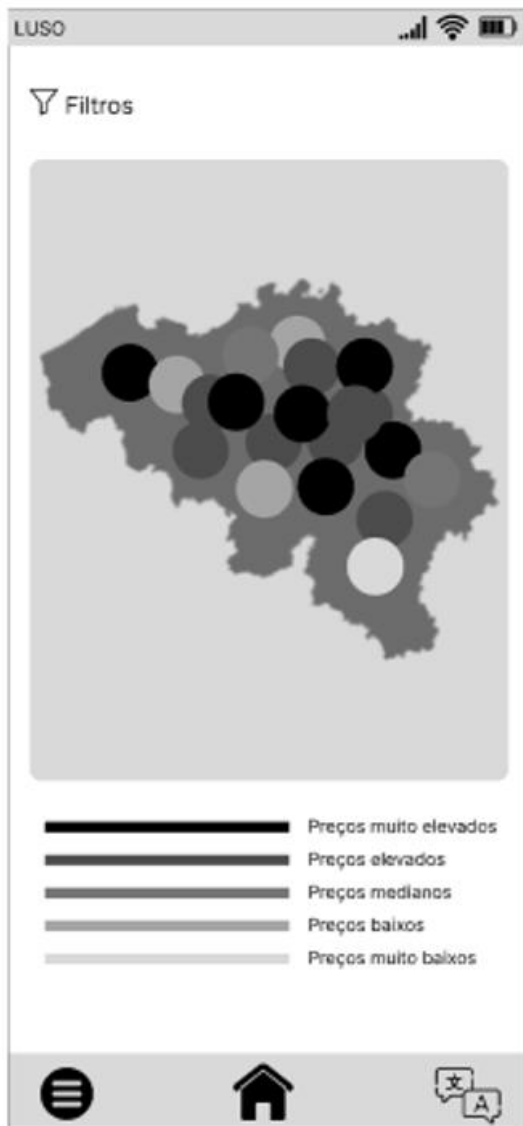


Figura 19 – Mockup 4

3.3.2 Protótipos de Interface de Alta-Fidelidade



Figura 20 – Mockup 5



Figura 21 – Mockup 6



Figura 22 – Mockup 7

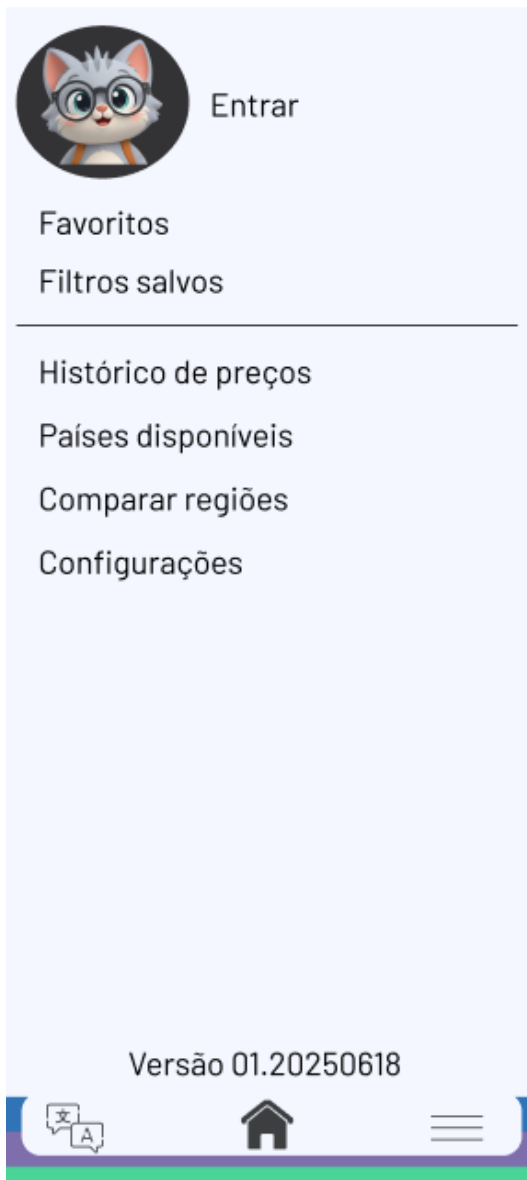


Figura 23 – Mockup 8



Figura 24 – Mockup 9

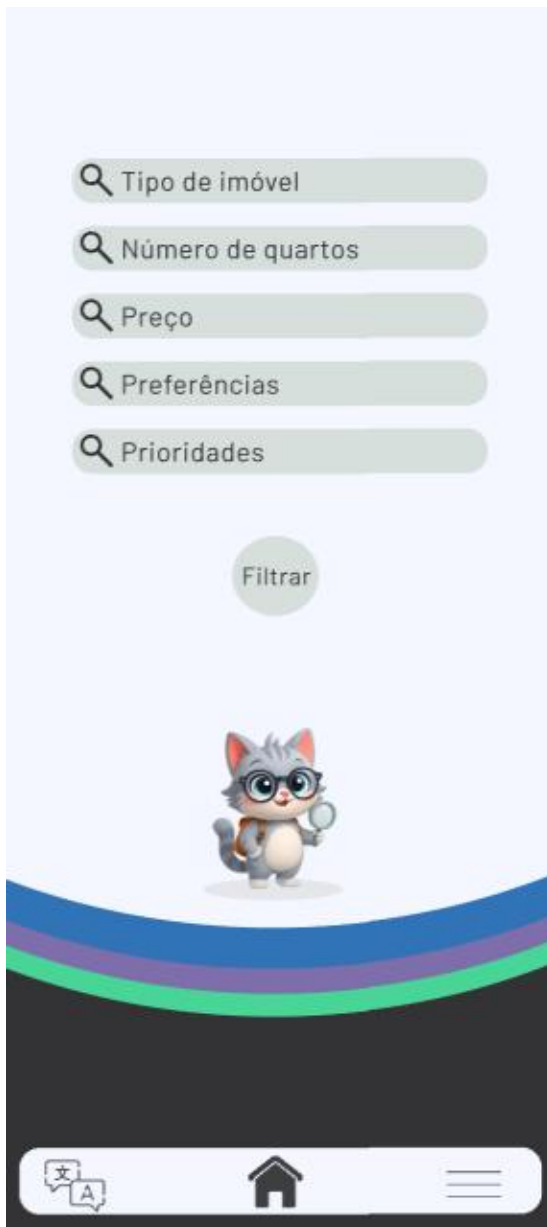


Figura 25 – Mockup 10

4 Solução Desenvolvida

4.1 Introdução

O RentScope é uma aplicação móvel desenvolvida com o objetivo de apoiar a tomada de decisão de utilizadores que procuram um local para morar com base em dados reais e comparáveis. O problema abordado decorre da dificuldade em analisar, de forma integrada, múltiplos fatores relevantes no contexto habitacional, como preços de arrendamento, disponibilidade de escolas, hospitais, criminalidade e outros indicadores territoriais. Esta dificuldade é particularmente acentuada quando o utilizador pretende mudar-se para uma região com a qual não possui familiaridade, seja ela um grande centro urbano ou uma zona menos povoada.

Para tornar esse processo mais acessível, a solução proposta utiliza uma representação cartográfica baseada em mapa coroplético, permitindo visualizar, de forma intuitiva, diferentes localidades e respetivas condições associadas à habitação. A aplicação procura transformar dados dispersos em informação útil, clara e visualmente interpretável, possibilitando ao utilizador identificar zonas potencialmente mais adequadas às suas preferências. A solução inclui ainda mecanismos de filtragem, visualização geográfica, integração com base de dados remota, autenticação, histórico de dados de renda e componentes interativos orientados à melhoria da experiência de utilização.

4.2 Arquitetura

A arquitetura do RentScope foi concebida segundo uma abordagem em camadas, separando claramente os processos de ingestão e tratamento de dados, o armazenamento remoto, a lógica de negócio e a interface da aplicação móvel. Esta organização permite maior modularidade, segurança, manutenção e possibilidade de evolução futura.

Numa primeira camada encontra-se o processo de ingestão de dados, realizado através de um script em Python com função de ETL (Extract, Transform, Load). Este componente é responsável por ler os ficheiros CSV, normalizar os dados e inseri-los na base de dados remota. O script não comunica diretamente com a aplicação móvel, sendo utilizado apenas para abastecimento e atualização da base de dados, podendo ser executado manualmente ou de forma agendada.

Na camada de persistência, foi adotado o Supabase como infraestrutura de base de dados remota baseada em PostgreSQL. Nesta camada ficam armazenados os dados oficiais processados, incluindo a informação relativa a municípios, renda, escolas, hospitais, criminalidade e outros elementos necessários ao funcionamento da aplicação.

Entre a aplicação móvel e a base de dados foi introduzida uma API, alojada no Render, que atua como camada intermediária de acesso à informação. Esta API tem como função proteger a base de dados, centralizar as regras de negócio, validar o acesso do utilizador quando necessário e devolver os dados em formatos adequados ao frontend, nomeadamente JSON e GeoJSON. Deste modo, o aplicativo nunca acede diretamente à base de dados nem necessita de conhecer credenciais sensíveis.

Na camada de apresentação encontra-se a aplicação Android, desenvolvida em Kotlin com Jetpack Compose. Esta aplicação consome os endpoints expostos pela API, processa as respostas recebidas e apresenta ao utilizador mapas, listas, gráficos e outras visualizações. Quando aplicável, também gere o processo de autenticação e envia o token correspondente nas requisições protegidas.

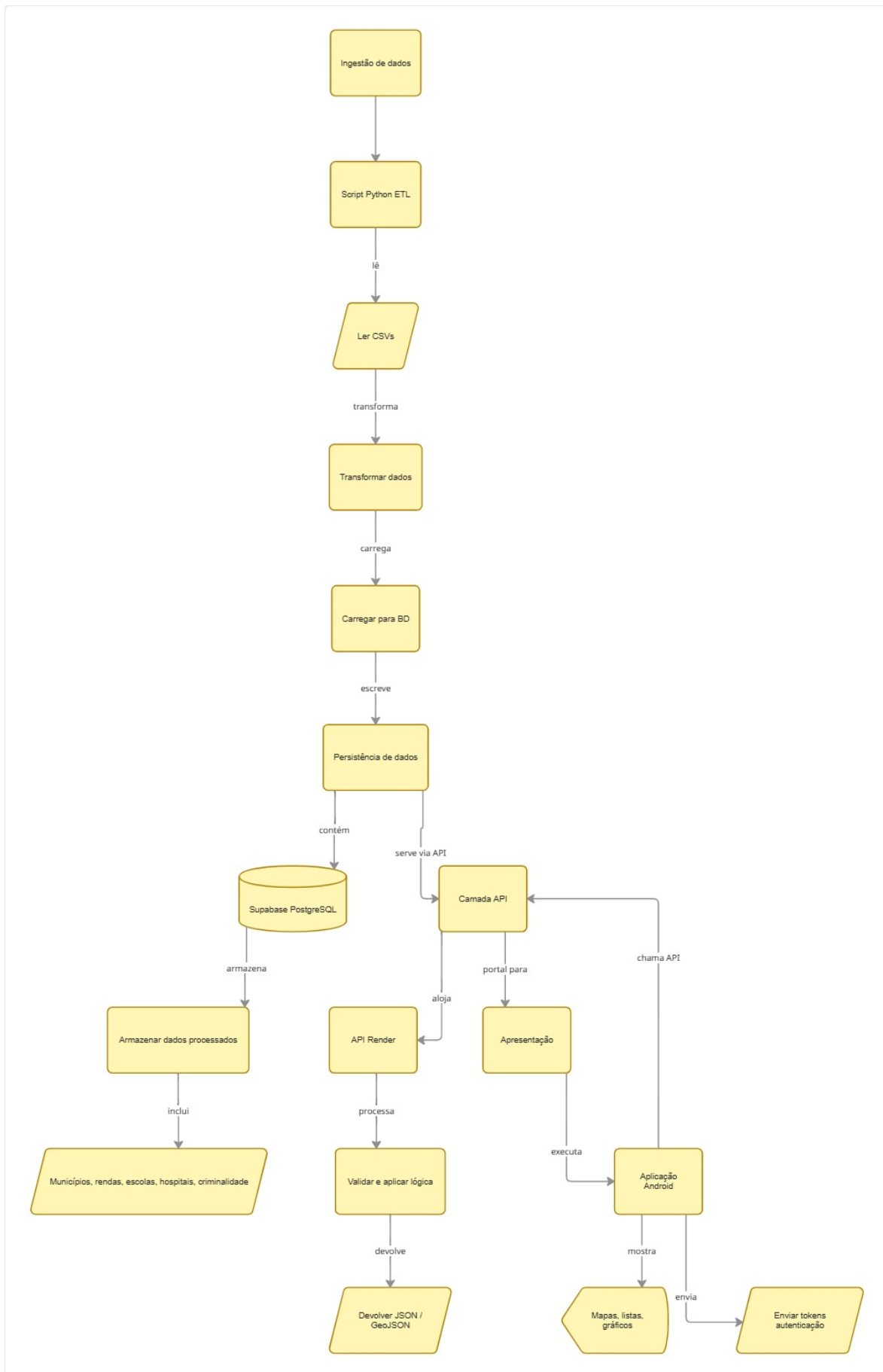


Figura 26 - Arquitetura do Software

4.3 Tecnologias e Ferramentas Utilizadas

A construção do RentScope envolveu a utilização de diferentes tecnologias, selecionadas de acordo com a sua adequação funcional, compatibilidade com os objetivos do projeto e alinhamento com boas práticas de desenvolvimento.

Os dados utilizados pela aplicação têm origem em ficheiros CSV, formato amplamente adotado por organismos públicos e institutos oficiais de estatística para disponibilização de dados abertos. A escolha deste formato justifica-se por ser simples, padronizado, independente de tecnologia e adequado ao tratamento de dados tabulares em larga escala. Além disso, trata-se do formato nativo em que os dados foram disponibilizados, o que evita transformações manuais adicionais e reduz o risco de perda de informação durante o processo de preparação.

Para o tratamento e normalização desses dados foi utilizada a linguagem Python. Esta escolha deve-se à sua ampla utilização em tarefas de processamento, limpeza e análise de dados, bem como à disponibilidade de bibliotecas especializadas para manipulação de ficheiros tabulares. A utilização de Python permitiu realizar tarefas de ETL, incluindo normalização de datas, valores numéricos, nomes de regiões e validação de inconsistências, preparando os dados para posterior inserção na base remota. No contexto do backend, foi igualmente utilizada a ferramenta Poetry para gestão das dependências e do ambiente do projeto.

Relativamente ao armazenamento, optou-se por uma base de dados remota em vez de armazenamento local. Esta decisão assegura centralização dos dados, consistência de acesso entre diferentes dispositivos, maior escalabilidade e melhor adequação a requisitos académicos que exigem soluções não dependentes de armazenamento local. Dentro desse contexto, foi selecionado o Supabase, por disponibilizar uma infraestrutura remota baseada em PostgreSQL, oferecendo robustez, compatibilidade com SQL, interface de gestão intuitiva, possibilidade de integração com extensões geográficas e um plano gratuito adequado ao contexto académico.

A comunicação entre a aplicação móvel e a base de dados é realizada por meio de uma API. A introdução desta camada justifica-se pela necessidade de separar responsabilidades entre frontend e backend, aumentar a segurança, centralizar regras de negócio, controlar o acesso aos dados e disponibilizar respostas num formato padronizado adequado ao consumo pela aplicação. Para o alojamento desta API foi escolhido o Render, por ser uma solução simples e eficaz para hospedar serviços web, permitindo integração com repositórios Git, deploy automático, obtenção de URL pública e clara separação entre camada de aplicação e camada de dados.

No desenvolvimento da aplicação móvel foi utilizada a linguagem Kotlin, oficialmente recomendada para o ecossistema Android. A sua adoção permitiu beneficiar de integração nativa com a plataforma, interoperabilidade com Java, sintaxe mais segura e concisa, e suporte completo às bibliotecas modernas do Android. A interface foi construída com Jetpack Compose, framework moderna para desenvolvimento declarativo de interfaces.

A comunicação HTTP entre a aplicação Android e a API foi implementada com Retrofit, biblioteca amplamente utilizada para integração com APIs REST. O Retrofit simplifica a definição dos endpoints consumidos pela aplicação e permite a conversão automática das respostas JSON em objetos Kotlin. No projeto, esta funcionalidade foi complementada por bibliotecas de serialização, nomeadamente Moshi e, no frontend atual, também Kotlinx Serialization, conforme a organização adotada nas diferentes fases da implementação.

Para a componente cartográfica, foi integrado o Google Maps SDK for Android, com recurso à biblioteca Google Maps Compose, de forma a permitir a incorporação de mapas interativos diretamente em interfaces desenvolvidas com Jetpack Compose. Esta integração tornou possível a renderização do mapa base e dos polígonos administrativos associados aos municípios. Para representar os limites administrativos, foi utilizado o formato GeoJSON, carregado localmente a partir da pasta assets da aplicação. Durante os testes, verificou-se a necessidade de simplificação geométrica dos ficheiros devido às limitações de memória, tendo sido utilizada a ferramenta Mapshaper para reduzir a complexidade dos polígonos sem comprometer significativamente a forma geral das regiões.

Relativamente ao ambiente de desenvolvimento, foram utilizados o Visual Studio Code, para tratamento de dados e API, e o Android Studio para a interface e seus componentes.

4.4 Ambientes de Teste e de Produção

Durante o desenvolvimento do RentScope foram considerados dois contextos de execução distintos: o ambiente local de desenvolvimento e teste, e o ambiente remoto correspondente à disponibilização online dos serviços.

No ambiente local, a API foi executada através do Uvicorn, que funciona como servidor ASGI responsável por disponibilizar a aplicação FastAPI numa porta local. A execução em modo de desenvolvimento permitiu validar endpoints, testar rotas diretamente no navegador e utilizar a documentação automática gerada pela FastAPI através do Swagger. Neste contexto, estiveram acessíveis, entre outros, o endpoint de documentação local e o endpoint de verificação de estado da API.

A aplicação Android foi testada em emulador, sendo a execução realizada através de uma ligação entre o VS Code e o Android Studio. Para que o emulador pudesse aceder à API local, foi utilizada a configuração de rede apropriada no cliente Android, permitindo encaminhar as requisições para o servidor em execução no computador de desenvolvimento.

No ambiente de produção, a API foi alojada no Render e a base de dados mantida no Supabase. Esta separação permitiu manter uma arquitetura modular, em que o Supabase assume a responsabilidade pela persistência dos dados e o Render pela execução da lógica do backend. Para o alojamento da API foi adotado um plano pago do Render, garantindo execução contínua do serviço sem suspensão em períodos de inatividade, o que assegura tempo de resposta consistente em todas as utilizações da aplicação. O Supabase, em plano gratuito, disponibiliza uma instância PostgreSQL na nuvem, autenticação, armazenamento básico e capacidade suficiente para o volume de dados do projeto académico.

No que diz respeito à disponibilização da aplicação móvel para testes em dispositivos físicos, foi utilizada a plataforma Diawi. Esta ferramenta permite gerar um link de instalação direta a partir do ficheiro APK, possibilitando a distribuição do aplicativo sem necessidade de publicação na Google Play Store. Através do Diawi, foi possível instalar e testar a aplicação em dispositivos reais, facilitando a validação da interface, comportamento e integração com os serviços remotos em ambiente próximo do utilizador final.

No que diz respeito aos recursos necessários, a solução depende de conectividade de rede para comunicação entre a aplicação, a API e a base de dados remota. Requer ainda capacidade de armazenamento para a base de dados, para ficheiros auxiliares como GeoJSON e para os artefactos do frontend e backend. As especificações quantitativas detalhadas de processamento, memória e largura de banda poderão ser acrescentadas posteriormente.

Github: <https://github.com/juliassantoss>

API online (Render): <https://rentscope-api-w1b7.onrender.com>

Supabase: <https://sqlsbsrhtjpjdscxoqbt.supabase.com>

Diawi:

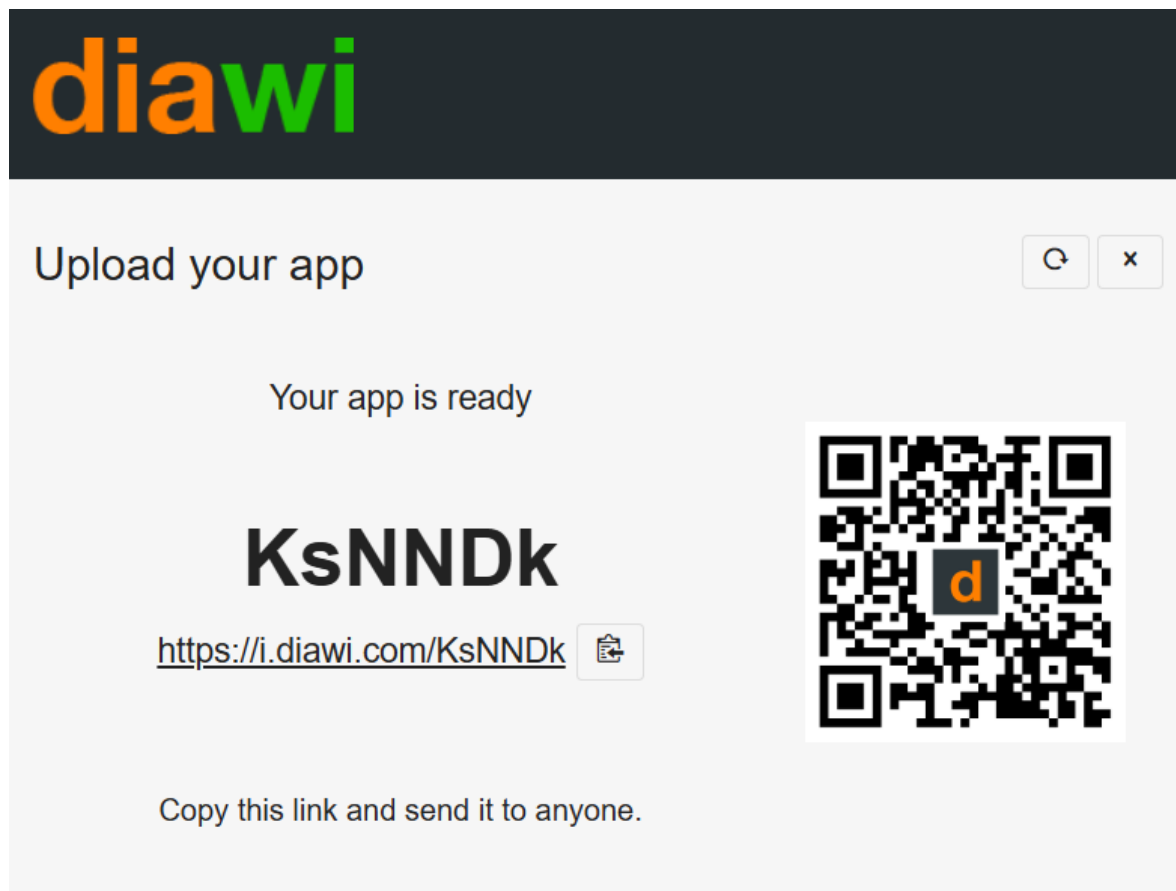
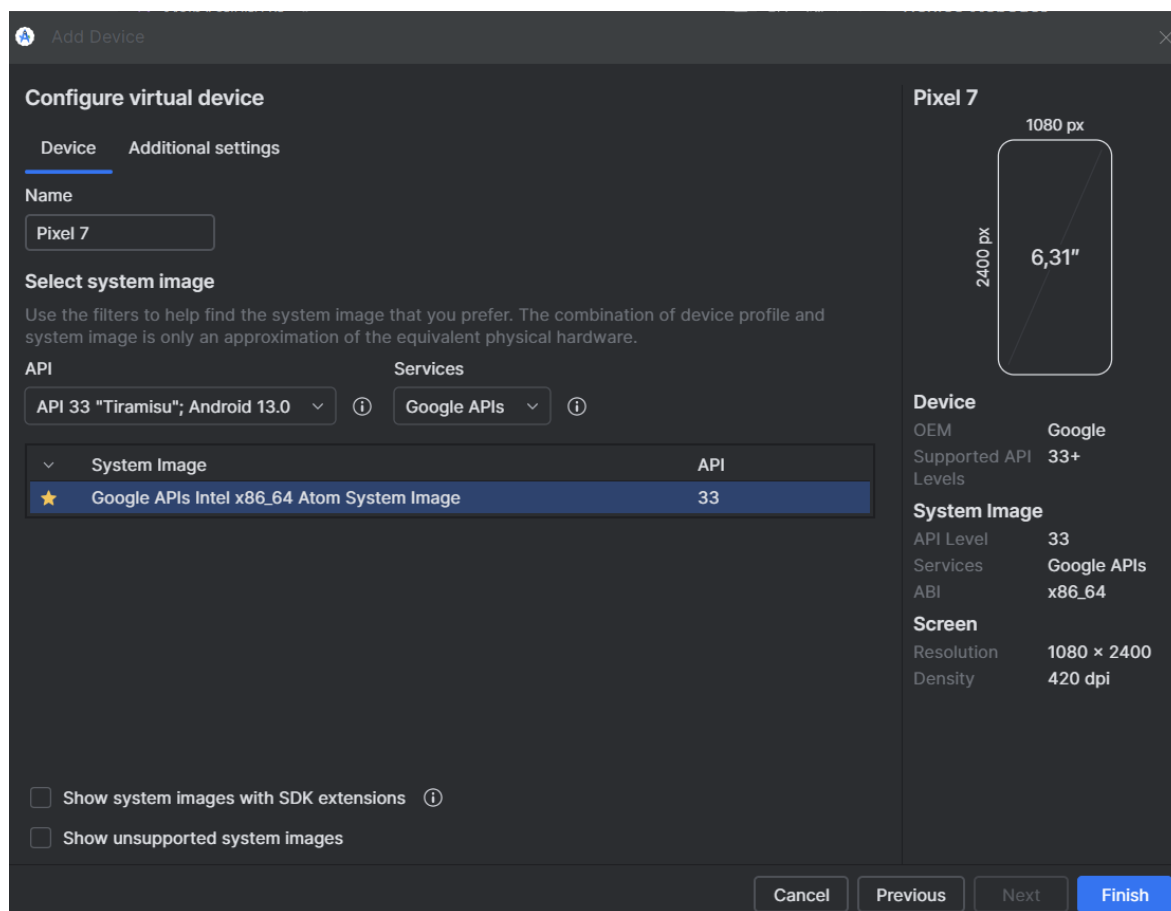


Figura 27 - QR code para baixar a aplicação

Obs.: o qrcode acima é válido somente por três dias depois de sua disponibilização, após esse período não é possível baixar o aplicativo através dele.

Configuração do emulador utilizado:



4.5 Abrangência

O desenvolvimento do RentScope mobiliza conhecimentos de diferentes áreas da formação em Engenharia Informática, articulando competências técnicas adquiridas ao longo do curso com a construção de uma solução aplicada a um problema concreto. Entre os domínios envolvidos destacam-se, em primeiro lugar, as bases de dados, através da modelação relacional, definição de tabelas, utilização de chaves de ligação e execução de consultas SQL sobre PostgreSQL.

A componente de desenvolvimento de software encontra-se presente na construção da aplicação Android, da API e dos scripts de processamento de dados, exigindo organização modular do código, separação de responsabilidades, manutenção e evolução incremental do sistema. A área de engenharia de software manifesta-se ainda na definição da arquitetura em camadas, na comunicação entre componentes, na integração entre frontend e backend e na adoção de práticas que favorecem escalabilidade e segurança.

A temática de sistemas distribuídos e serviços web também é contemplada, uma vez que a solução depende da interação entre uma aplicação cliente, uma API remota e uma base de dados na nuvem, com troca de informação através de protocolos HTTP e formatos padronizados como JSON e GeoJSON. De igual modo, a área de visualização de informação e interação humano-

computador é aplicada na construção do mapa coroplético, na organização da interface e na reformulação do design com base em princípios do Material 3.

Por fim, o projeto incorpora técnicas de tratamento e integração de dados através de pipelines ETL e da normalização de ficheiros CSV provenientes de fontes públicas, bem como mecanismos de segurança e autenticação, incluindo hashing de palavras-passe, verificação de e-mail e utilização de tokens JWT. A aplicação recolhe apenas os dados estritamente necessários ao seu funcionamento, nomeadamente e-mail, palavra-passe cifrada e preferências do utilizador, adotando práticas alinhadas com o Regulamento Geral sobre a Proteção de Dados (RGPD), como minimização de dados, armazenamento seguro e limitação de acesso às informações pessoais. Como trabalho futuro, prevê-se a disponibilização de uma política de privacidade e mecanismos de exportação e eliminação de conta.

4.6 Componentes

4.6.1 Componente 1 – Dados, ETL e Base de Dados

A componente de dados do RentScope foi estruturada a partir de ficheiros CSV previamente tratados através de pipelines ETL. Estes ficheiros correspondem a diferentes dimensões relevantes para a análise dos municípios, nomeadamente renda de arrendamento, número de escolas, número de hospitais e criminalidade. A primeira decisão estrutural consistiu em definir o município como entidade central do modelo de dados, através da criação de uma tabela dedicada que funciona como dimensão principal e é referenciada pelas restantes tabelas por meio do campo `codigo_municipio`.

A importação dos dados foi realizada diretamente no Supabase, respeitando a dependência entre tabelas. Inicialmente foi importada a tabela de municípios, seguida pelas tabelas factuais associadas aos diferentes indicadores. Para simplificar o processo de integração, os nomes das colunas na base de dados foram alinhados com os nomes presentes nos ficheiros CSV finais, reduzindo a necessidade de transformações adicionais no momento da carga.

A tabela de renda foi estruturada com natureza temporal, armazenando registos por município e por trimestre, o que permite representar a evolução dos preços desde 2022 até ao primeiro trimestre de 2025. As tabelas de escolas, hospitais e criminalidade foram importadas como valores agregados por município, uma vez que o objetivo da aplicação é utilizar esses indicadores como fatores de análise territorial e não como registos individualizados.

Durante a validação dos dados importados foi identificado um problema na tabela de criminalidade, em que o número de municípios presentes no CSV era inferior ao esperado face ao ficheiro original em formato XLS. A análise da pipeline revelou que estavam a ser excluídos municípios com códigos de sete dígitos, por se considerar apenas códigos com seis dígitos. Essa limitação foi corrigida no processo de transformação, o CSV foi regenerado e a importação repetida, assegurando maior cobertura dos municípios.

Durante a integração dos dados de renda foi identificada uma característica relevante da fonte: o INE publica preços medianos de arrendamento ao nível do município apenas para os concelhos com mais de 100 mil habitantes, num total de 24 sobre os 308 existentes em Portugal. Esta restrição decorre da exigência de volume estatístico mínimo para garantir representatividade dos valores publicados. Optou-se por preservar esta decisão metodológica e não recorrer a imputação

ou estimativa para os restantes municípios, sinalizando de forma transparente, na interface, a ausência de informação.

Com os dados já estruturados, tornou-se possível construir consultas no backend que agregam, para cada município, os valores mais recentes dos diferentes indicadores. Nos casos em que existem séries temporais, como renda e escolas, foram utilizadas funções de janela para selecionar automaticamente o registo mais recente por município.

4.6.2 Componente 2 – API e Integração com a Base de Dados

A API do RentScope foi desenvolvida em FastAPI e desempenha o papel de camada intermediária entre a aplicação Android e a base de dados remota. Esta opção arquitetural permite proteger o acesso à base, centralizar a lógica de negócio e fornecer respostas num formato padronizado adequado ao consumo pela aplicação.

A integração entre a API e o Supabase foi realizada através da biblioteca `psycopg`, permitindo executar consultas SQL diretamente sobre PostgreSQL. A ligação foi configurada com base na `DATABASE_URL` fornecida pelo Supabase, incluindo suporte a SSL sempre que necessário. Para garantir a correta serialização das respostas, os resultados das consultas foram convertidos para estruturas compatíveis com JSON antes de serem devolvidos ao cliente.

A organização interna do backend foi dividida em módulos, incluindo ficheiros responsáveis pela configuração, ligação à base de dados, definição da aplicação principal e routers separados por recurso. Esta estrutura favorece a legibilidade do projeto e facilita a manutenção. A validação inicial da integração foi efetuada através do endpoint `/países`, que consultou corretamente a tabela respetiva e devolveu os dados estruturados para consumo pela aplicação Android.

No backend foi também implementado um sistema de autenticação próprio, com o objetivo de garantir controlo de acesso e segurança. As palavras-passe são armazenadas sob a forma de hash com `pbkdf2_sha256`, evitando qualquer persistência em texto simples. Foi criada uma tabela de utilizadores contendo informações como identificador, e-mail, estado de verificação e data de criação. O processo de registo inclui validação dos dados e rejeição de e-mails duplicados.

Após o registo, foi implementado um mecanismo de verificação de e-mail com geração de token temporário e envio de mensagem real por SMTP. O processo de login recorre a JSON Web Tokens, com emissão de `access_token` e `refresh_token`, permitindo autenticação stateless e renovação segura de sessão. Foram ainda disponibilizados endpoints protegidos para identificação do utilizador autenticado e atualização da sessão.

4.6.3 Componente 3 - Aplicação Android e Consumo API

A aplicação móvel foi desenvolvida em Kotlin e Jetpack Compose, assumindo a responsabilidade pela interface com o utilizador e pelo consumo dos serviços disponibilizados pela API. A comunicação HTTP foi implementada com Retrofit, que permite definir interfaces correspondentes aos endpoints e converter automaticamente as respostas JSON em objetos Kotlin. Nas primeiras fases da integração, a comunicação foi validada a partir do endpoint `/países`, confirmando a ligação funcional entre Android, FastAPI e Supabase.

A organização do frontend foi estruturada em camadas, incluindo DTOs, cliente HTTP, interface da API, repositórios e ViewModels. Esta separação permitiu isolar a lógica de acesso a dados da

lógica de apresentação, tornando o código mais organizado e mais simples de evoluir. A utilização de StateFlow e coroutines permitiu gerir estados de carregamento, erro e sucesso de forma reativa.

Inicialmente, foi criada uma tela de depuração para validar a comunicação com o backend, exibindo o estado da chamada e a lista de países devolvida pela API. Posteriormente, essa integração foi aplicada numa tela funcional da aplicação, substituindo dados simulados por dados reais obtidos do servidor. Esta etapa marcou a transição do protótipo estático para uma solução efetivamente integrada com backend.

4.6.4 Componente 4 – Visualização Geográfica com Google Maps e GeoJSON

A componente cartográfica do RentScope foi implementada com o Google Maps SDK for Android, utilizando a biblioteca Google Maps Compose para integração com Jetpack Compose. Esta escolha permitiu incorporar mapas interativos diretamente na interface da aplicação e criar a base necessária para a representação espacial dos dados territoriais.

Para suportar a visualização dos limites administrativos, foi integrado um ficheiro GeoJSON armazenado localmente na pasta assets da aplicação. O carregamento deste ficheiro é realizado por um componente responsável por ler o seu conteúdo e disponibilizá-lo para processamento. Em seguida, o parser extrai as features geográficas, interpreta as respetivas propriedades e transforma as coordenadas em estruturas compatíveis com o mapa, permitindo desenhar polígonos correspondentes aos municípios.

O ficheiro GeoJSON utilizado foi derivado da CAOP, contendo os limites administrativos oficiais dos concelhos. Cada feature inclui informação geométrica e metadados, como nome do concelho e código identificador. Estes metadados são essenciais para associar cada polígono aos dados analíticos da aplicação. Durante os testes, o dataset inicial revelou problemas de consumo de memória devido ao elevado número de vértices. Para resolver essa limitação, o ficheiro foi simplificado com a ferramenta Mapshaper, permitindo manter a representação dos municípios sem provocar falhas de execução.

Esta infraestrutura tornou possível preparar a base para o mapa coroplético, em que cada município pode receber uma cor associada a um valor numérico. A lógica de visualização das cores baseia-se na atribuição de um score normalizado entre 0 e 1 para cada município, representando o grau de adequação às preferências do utilizador. Esse score é depois convertido numa cor aplicada ao polígono respetivo, permitindo identificar visualmente, no mapa, as zonas mais ou menos adequadas segundo os critérios analisados.

4.6.5 Componente 5 – Mascote Interativo

Com o objetivo de melhorar a experiência de utilização e reforçar a identidade visual da aplicação, foi desenvolvido um mascote interativo integrado diretamente na interface do RentScope.

O mascote foi construído como um componente visual animado, em Jetpack Compose, representado por uma esfera com efeitos de brilho, reflexo e flutuação suave. Foram ainda adicionados elementos visuais simples, como olhos, de forma a conferir uma aparência mais expressiva e amigável. O comportamento do mascote é controlado por diferentes estados visuais, associados a situações como inatividade, processamento, resposta e erro.

A integração deste componente com a lógica do assistente inteligente permite que o mascote reaja em tempo real às ações do utilizador, nomeadamente quando é realizada uma pergunta ou quando uma resposta for recebida. O mascote é apresentado na página inicial da aplicação, acompanhado de um balão de fala e de um botão de acesso ao assistente. Assim, deixa de ser apenas um elemento decorativo e passa a desempenhar também uma função de orientação e mediação da interação.

4.6.6 Componente 6 – Assistente Inteligente

Como forma de acrescentar uma camada interpretativa à informação apresentada na aplicação, foi integrada no RentScope uma funcionalidade de assistente inteligente baseada em inteligência artificial. O objetivo desta componente consiste em permitir ao utilizador colocar questões em linguagem natural sobre um determinado local, obtendo respostas curtas, úteis e contextualizadas sobre a adequação de um município ou país para habitação. Esta funcionalidade procura complementar a análise quantitativa disponibilizada pelos indicadores e pelo mapa, oferecendo uma explicação textual que facilite a compreensão dos dados e apoie a tomada de decisão.

A arquitetura adotada para esta funcionalidade segue o mesmo princípio de separação por camadas presente no restante sistema. A aplicação Android não comunica diretamente com o serviço de inteligência artificial. Em vez disso, envia o pedido para o backend desenvolvido em FastAPI, que atua como intermediário entre o frontend e a API da OpenAI. Esta opção permite proteger a chave de acesso, manter credenciais sensíveis fora da aplicação móvel e centralizar o controlo da lógica associada às perguntas e respostas geradas.

No backend, foi criado um endpoint específico para receção das perguntas do utilizador. Esse endpoint recebe informação como o país, o município, a pergunta formulada e, quando disponível, valores associados a indicadores relevantes, como renda, escolas, hospitais, criminalidade e score geral. A partir desses dados, o servidor constrói o contexto enviado ao modelo de linguagem, permitindo que a resposta gerada tenha em consideração a informação disponível no sistema. A resposta devolvida ao frontend é depois apresentada ao utilizador de forma integrada na interface da aplicação.

Do ponto de vista funcional, esta componente permite responder a questões como a qualidade de vida esperada num determinado local, os principais pontos positivos e negativos de um município, a adequação de uma região para famílias ou uma interpretação geral do score atribuído pelo sistema. Desta forma, a inteligência artificial não substitui os dados objetivos da aplicação, mas atua como um mecanismo complementar de explicação e interpretação.

A integração no frontend foi realizada através de componentes próprios no Android, incluindo uma screen dedicada ao assistente e uma ViewModel responsável por gerir o estado da interação. Durante o ciclo de utilização, o sistema apresenta diferentes estados, como carregamento, resposta recebida e erro, permitindo uma experiência de utilização mais clara e controlada. A comunicação entre frontend e backend segue o mesmo padrão adotado para os restantes serviços da aplicação, mantendo coerência arquitetural.

Adicionalmente, esta funcionalidade foi articulada com o mascote interativo presente na interface inicial da aplicação. O mascote reage visualmente ao estado da interação com a inteligência artificial, assumindo comportamentos distintos quando está inativo, a processar uma pergunta, a apresentar uma resposta ou quando ocorre uma falha. Esta integração reforça a percepção de resposta do sistema e contribui para tornar a experiência mais intuitiva e envolvente.

Assim, a componente de assistente inteligente representa uma extensão funcional relevante do RentScope, ao combinar análise de dados estruturados com geração de respostas em linguagem natural. Esta abordagem contribui para tornar a aplicação não apenas uma ferramenta de visualização e comparação, mas também um sistema capaz de apoiar o utilizador com explicações mais acessíveis, contextualizadas e próximas da forma como as pessoas efetivamente colocam dúvidas no processo de escolha de um local para viver.

4.7 Interfaces

Houve uma atualização do design da aplicação em relação aos mockups previamente apresentados, essa modificação foi orientada pela adoção do Material 3, atualmente recomendado pelo Google para aplicações Android. Esta decisão permitiu modernizar a interface, melhorar a hierarquia visual da informação, reduzir excessos decorativos e facilitar a implementação técnica, uma vez que os componentes do Material 3 possuem correspondência direta com os elementos disponibilizados pelo Jetpack Compose.



Figura 28 - Home Screen



Figura 29 - IA Screen

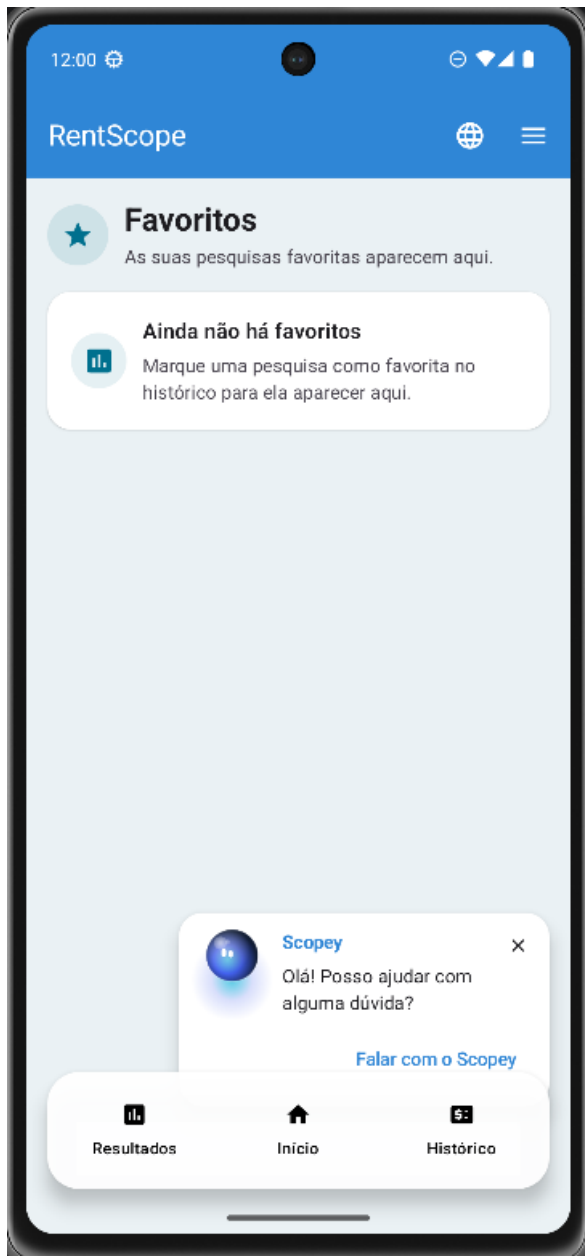


Figura 30 - Favorites Screen

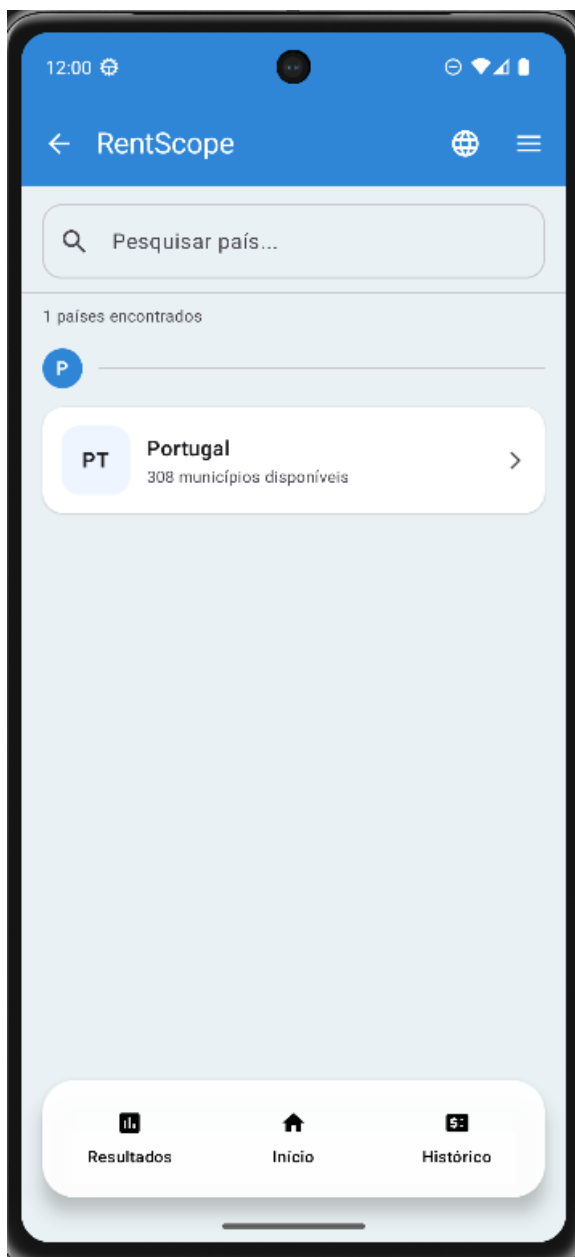


Figura 31 - Search Country Screen

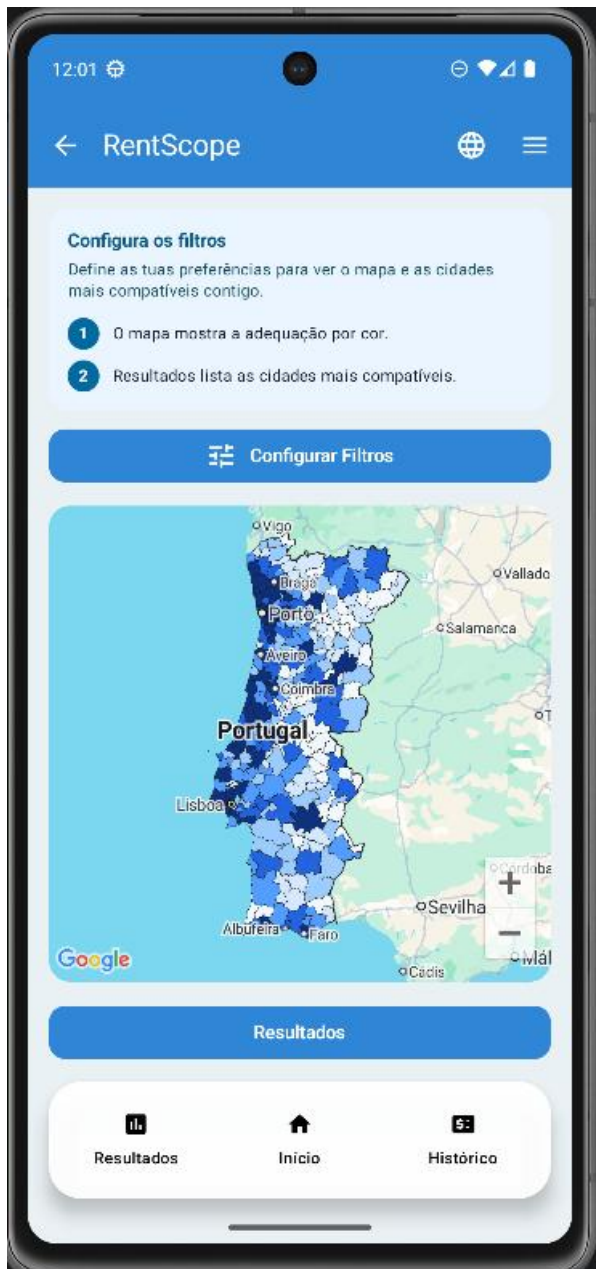


Figura 32 - Map Screen

5 Testes e Validação

5.1 Abordagem e Justificação dos Testes

O plano de testes do RentScope foi concebido com o objetivo de validar de forma estruturada o funcionamento técnico da aplicação, a integração entre os seus componentes e a sua aplicabilidade prática enquanto solução para apoio à decisão na escolha de um local para habitação.

A abordagem adotada baseia-se numa validação progressiva por níveis, permitindo testar o sistema desde os seus componentes individuais até à sua utilização em contexto real. Esta estratégia foi escolhida de forma a reduzir riscos associados à integração entre tecnologias distintas e garantir que eventuais falhas são identificadas e corrigidas em fases iniciais do desenvolvimento.

Os testes foram organizados em diferentes categorias, nomeadamente testes funcionais, testes de integração, testes de validação operacional e testes de usabilidade. Esta divisão permite avaliar não apenas se o sistema funciona corretamente, mas também se cumpre os objetivos propostos, nomeadamente a capacidade de apresentar informação clara, comparável e útil para o utilizador.

Adicionalmente, foi considerada a análise de riscos associados ao sistema, tendo em conta fatores como dependência de serviços externos, conectividade de rede e processamento de dados geográficos.

5.2 Ambientes de Teste

Em uma primeira fase, foram realizados testes em ambiente local de desenvolvimento, onde a aplicação Android foi executada em emulador e a API foi executada localmente através do Uvicorn. Este ambiente permitiu validar funcionalidades de forma iterativa e realizar correções rápidas durante o desenvolvimento.

Posteriormente, os testes foram realizados em ambiente remoto, com a API alojada no Render e a base de dados no Supabase. Este contexto permitiu validar a comunicação entre os diferentes componentes do sistema em condições reais de rede e com serviços cloud ativos.

Por fim, foram realizados testes em dispositivo físico, através da instalação da aplicação num telemóvel utilizando a plataforma Diawi. Esta abordagem permitiu validar o comportamento da aplicação fora do ambiente de desenvolvimento, incluindo aspetos relacionados com desempenho, conectividade e experiência de utilização.

5.3 Testes Funcionais

Os testes funcionais tiveram como objetivo validar o correto funcionamento das funcionalidades principais da aplicação, garantindo que cada componente cumpre o comportamento esperado.

Foram definidos cenários de teste para as principais operações da aplicação, incluindo autenticação de utilizadores, navegação entre ecrãs, seleção de país e município, visualização do mapa, aplicação de filtros, consulta de dados históricos, gestão de favoritos e utilização do assistente inteligente.

Cada cenário de teste foi estruturado com base em passos de execução e resultados esperados, permitindo verificar de forma sistemática se a aplicação responde corretamente às ações do utilizador.

5.4 Testes de Integração

Os testes de integração tiveram como objetivo validar a comunicação entre os diferentes componentes do sistema, assegurando o correto funcionamento da arquitetura em camadas adotada.

Foram testados os fluxos de comunicação entre a aplicação Android, a API desenvolvida em FastAPI e a base de dados no Supabase, bem como a integração com serviços externos, nomeadamente o Google Maps e a API de inteligência artificial.

Estes testes permitiram verificar a consistência dos dados, o correto processamento das requisições e a segurança na comunicação, nomeadamente através da utilização de tokens de autenticação e da proteção de credenciais sensíveis no backend.

5.5 Testes de Validação Operacional

Os testes de validação operacional foram definidos com o objetivo de verificar o funcionamento da aplicação em condições próximas de um ambiente real de utilização.

Estes testes incluíram a execução da aplicação em dispositivo físico, a comunicação com serviços remotos e a utilização das funcionalidades principais em contexto real. Foram também considerados cenários de falha, como ausência de conectividade de rede ou indisponibilidade temporária da API, de forma a avaliar a robustez do sistema.

A realização destes testes permitiu validar a operacionalidade da solução e a sua capacidade de funcionar de forma consistente fora do ambiente de desenvolvimento.

5.6 Testes de Usabilidade

Os testes de usabilidade foram definidos com o objetivo de avaliar a facilidade de utilização da aplicação e a clareza na apresentação da informação.

Para este efeito, foram definidos cenários de utilização representativos, nos quais os utilizadores deveriam realizar tarefas típicas, como selecionar um país, interpretar o mapa, aplicar filtros e consultar informações adicionais.

Foram considerados critérios como facilidade de navegação, compreensão da interface, clareza dos dados apresentados e utilidade da aplicação para o objetivo proposto. A recolha de feedback foi realizada através de observação direta e de questões orientadas, permitindo obter uma avaliação qualitativa da experiência de utilização.

5.7 Análise de Riscos e Impacto

No contexto do plano de testes, foi realizada uma análise de riscos com o objetivo de identificar potenciais falhas que possam afetar o funcionamento do sistema.

Entre os principais riscos identificados destacam-se a dependência de serviços externos, a possibilidade de falhas de conectividade de rede, o processamento de grandes volumes de dados geográficos e a consistência dos dados provenientes de fontes externas.

Para mitigar estes riscos, foram adotadas diversas estratégias, incluindo a utilização de uma arquitetura em camadas, a implementação de mecanismos de tratamento de erro na interface, a simplificação dos dados geográficos e a centralização da lógica de acesso a serviços externos no backend.

Esta análise permitiu orientar os testes para cenários críticos e contribuir para a robustez e fiabilidade da solução.

6 Método e Planeamento

6.1 Metodologia de Desenvolvimento

O desenvolvimento do RentScope seguiu uma abordagem iterativa e incremental, permitindo a evolução progressiva da solução ao longo do tempo. Em vez de uma implementação linear, o projeto foi dividido em fases distintas, cada uma focada em componentes específicos do sistema, como a base de dados, backend, aplicação Android e funcionalidades avançadas como visualização geográfica e integração com inteligência artificial.

Esta abordagem permitiu validar cada componente individualmente antes da sua integração, reduzindo riscos e facilitando a identificação de erros em fases iniciais do desenvolvimento. A comunicação entre componentes foi introduzida gradualmente, começando com dados estáticos e evoluindo para integração completa com serviços remotos.

6.2 Plano de Trabalho e Cronograma

O plano de trabalho foi estruturado com base nas fases principais do projeto, considerando uma progressão desde a preparação dos dados até à validação final da aplicação.

De forma geral, o desenvolvimento seguiu as seguintes etapas:

- definição do problema e levantamento de requisitos;
- recolha e tratamento de dados (CSV e ETL em Python);
- modelação da base de dados em PostgreSQL (Supabase);
- desenvolvimento da API em FastAPI;
- desenvolvimento da aplicação Android;
- integração com serviços externos (Google Maps e OpenAI);
- implementação de funcionalidades avançadas (mapa coroplético, IA, histórico);
- testes e validação em ambiente real;
- preparação para disponibilização da solução.

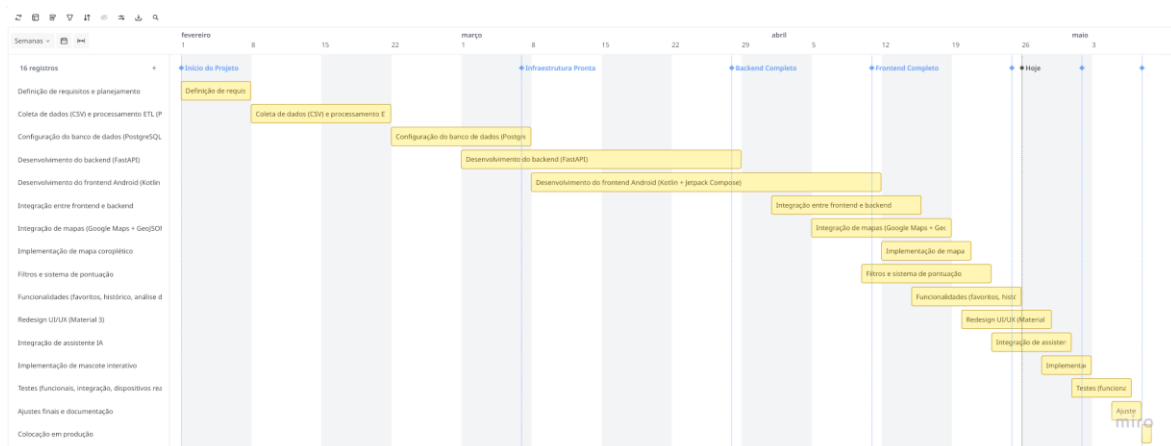


Figura 33 - Diagrama de Gantt

O cronograma final foi ajustado considerando uma hipotética disponibilização pública da aplicação.

6.3 Análise Crítica do Cumprimento do Plano

De forma geral, o plano de trabalho foi cumprido, embora com alguns ajustes ao longo do desenvolvimento. Algumas tarefas inicialmente previstas sofreram alterações de prioridade, especialmente com a introdução de funcionalidades adicionais.

Entre as principais dificuldades encontradas destacam-se:

- tratamento e normalização de dados provenientes de diferentes fontes;
- gestão de desempenho na renderização de ficheiros GeoJSON de grande dimensão;
- integração entre múltiplas tecnologias (Android, API, base de dados e serviços externos);
- configuração e segurança de chaves de acesso a serviços externos;
- cobertura parcial dos dados oficiais de renda, restritos aos 24 concelhos com mais de 100 mil habitantes, conforme critério do INE.

Estas dificuldades levaram à necessidade de adaptação do plano inicial, incluindo:

- simplificação dos dados geográficos para melhorar desempenho;
- introdução de uma camada de API para maior controlo e segurança;
- reorganização da interface para melhorar a experiência do utilizador;
- ajuste da ordem de implementação das funcionalidades.

Apesar destas adaptações, o progresso do projeto manteve-se consistente, sendo possível atingir os objetivos principais definidos inicialmente.

7 Resultados dos Testes

7.1 Descrição dos Resultados

Os testes realizados ao longo do desenvolvimento permitiram validar o funcionamento da aplicação em diferentes níveis, incluindo funcionalidade, integração e operação em ambiente real.

No que diz respeito aos testes funcionais, verificou-se que todas as funcionalidades principais da aplicação operam conforme esperado, incluindo autenticação, navegação, visualização do mapa coroplético, aplicação de filtros, histórico de dados e utilização do assistente inteligente.

Relativamente aos testes de integração, foi confirmada a correta comunicação entre os diferentes componentes do sistema, nomeadamente a aplicação Android, a API desenvolvida em FastAPI e a base de dados no Supabase. O fluxo de dados entre frontend e backend demonstrou consistência, garantindo que as informações apresentadas ao utilizador correspondem aos dados armazenados.

No contexto de validação operacional, a aplicação foi testada em dispositivo físico através da instalação via Diawi, utilizando a API alojada no Render. Estes testes permitiram confirmar o funcionamento da solução em ambiente real, incluindo o comportamento da aplicação com conectividade de rede, serviços cloud e execução fora do ambiente de desenvolvimento.

Adicionalmente, foram realizados testes com utilizadores externos, permitindo avaliar a usabilidade e a aplicabilidade prática da solução. Os resultados indicam que os utilizadores conseguiram utilizar a aplicação de forma intuitiva, compreendendo a informação apresentada e identificando utilidade na comparação entre municípios.

7.1.2 Outputs e Outcomes

Os principais outputs da solução correspondem à apresentação visual e interativa dos dados, incluindo:

- mapa coroplético com representação dos municípios;
- ranking de localidades com base em score;
- gráficos de evolução de preços;
- respostas geradas pelo assistente inteligente.

Os outcomes observados incluem:

- melhoria na compreensão dos dados por parte dos utilizadores;
- capacidade de comparação entre diferentes localidades;
- apoio à tomada de decisão com base em múltiplos fatores;
- utilização de linguagem natural para interpretação dos resultados.

7.1.3 Validação por Terceiros

A validação com utilizadores externos demonstrou que a aplicação apresenta boa usabilidade e clareza na apresentação da informação. Os participantes conseguiram executar as tarefas propostas sem dificuldades significativas, destacando o mapa e o assistente inteligente como elementos facilitadores da interpretação dos dados.

Os resultados detalhados dos testes, incluindo cenários e observações, encontram-se descritos no Anexo A.

7.2 Cumprimentos de Requisitos

7.2.1 Requisitos Funcionais

Requisito	Estado	Justificação
REQ.001 – Página de Login	Realizado	Sistema de autenticação implementado com registo, login e verificação de e-mail
REQ.002 – Mapa Coroplético Interativo	Realizado	Implementado com Google Maps e GeoJSON, incluindo representação visual baseada em score
REQ.003 – Filtros Personalizados	Realizado	Sistema de filtros contínuos baseado em múltiplos indicadores
REQ.004 – Comparação de Localidades	Realizado	Implementada uma screen que permite visualizar e comparar localidades lado a lado
REQ.005 – Análise Temporal	Realizado	Histórico de preços implementado com visualização gráfica

REQ.006 – Perfil do Utilizador	Parcialmente realizado	Funcionalidade de favoritos implementada; alertas não foram desenvolvidos devido à limitação dos dados disponíveis, que não apresentavam atualização suficientemente frequente para justificar a sua implementação
REQ.007 – Mascote Interativo	Realizado	Mascote implementado com diferentes estados e integração com o assistente
REQ.008 – Gestão da Base de Dados	Realizado	Base de dados estruturada e gerida em ambiente cloud (Supabase)
REQ.009 – Interface de Navegação	Realizado	Interface desenvolvida em Jetpack Compose com navegação interativa e consistente

Tabela 17 - Cumprimento dos Requisitos Funcionais

7.2.2 Requisitos não-funcionais

Requisito	Estado	Justificação
REQNF.001 – Usabilidade	Realizado	Interface baseada em Material 3 e validada através de testes de usabilidade
REQNF.002 – Confiabilidade dos Dados	Realizado	Dados provenientes de fontes oficiais e tratados através de pipelines ETL
REQNF.003 – Segurança	Parcialmente realizado	Sistema de autenticação implementado; autenticação multifatorial não foi incluída
REQNF.004 – Desempenho	Realizado	Otimizações realizadas, incluindo simplificação dos dados geográficos
REQNF.005 – Disponibilidade	Realizado	API alojada em plano pago do Render para garantir disponibilidade contínua, sem suspensão em períodos de inatividade
REQNF.006 – Escalabilidade	Realizado	Arquitetura em camadas permite expansão futura do sistema

Tabela 18 - Cumprimento dos Requisitos Não-Funcionais

7.3 Funcionalidades Adicionais e Melhorias Introduzidas

Ao longo do desenvolvimento do RentScope, para além dos requisitos inicialmente definidos, foram introduzidas funcionalidades adicionais com o objetivo de enriquecer a experiência do utilizador e aumentar o valor prático da aplicação.

Uma das principais extensões ao projeto foi a integração de um assistente inteligente baseado em inteligência artificial. Esta funcionalidade permite ao utilizador colocar questões em linguagem natural sobre determinados locais, obtendo respostas interpretativas que complementam os dados quantitativos apresentados pela aplicação. Esta abordagem contribui para tornar a análise mais acessível, especialmente para utilizadores sem experiência na interpretação de dados estatísticos.

Outra melhoria relevante foi a reformulação do design da aplicação, com adoção do sistema Material 3, permitindo uma interface mais moderna, consistente e alinhada com as boas práticas do ecossistema Android. Esta alteração contribuiu significativamente para a usabilidade e clareza da aplicação.

Foram também introduzidas melhorias ao nível da visualização de dados, incluindo a implementação de gráficos para análise temporal e otimizações na renderização do mapa, nomeadamente através da simplificação dos ficheiros GeoJSON, garantindo melhor desempenho e estabilidade.

Estas funcionalidades adicionais não só complementam os requisitos inicialmente definidos, como também reforçam a utilidade e diferenciação da solução, aproximando-a de um produto com maior maturidade e aplicabilidade prática.

8 Conclusão

O desenvolvimento do RentScope permitiu a construção de uma solução funcional orientada ao apoio à tomada de decisão na escolha de um local para habitação, através da integração de dados reais, visualização geográfica e mecanismos de análise interativa. Ao longo do projeto, foi possível implementar uma aplicação que combina diferentes componentes tecnológicos, incluindo processamento de dados, base de dados remota, API, aplicação móvel e serviços externos, resultando numa arquitetura coerente e alinhada com práticas atuais de desenvolvimento de software.

A solução desenvolvida cumpre, de forma geral, os objetivos propostos, disponibilizando ao utilizador ferramentas que permitem analisar e comparar diferentes localidades com base em múltiplos indicadores relevantes. A utilização de um mapa coroplético, associada a um sistema de scoring contínuo, demonstrou ser eficaz na apresentação de informação complexa de forma acessível e intuitiva. Adicionalmente, funcionalidades como o histórico de preços, a comparação de localidades e o assistente inteligente contribuem para enriquecer a experiência de utilização e facilitar a interpretação dos dados.

Os testes realizados, tanto em ambiente controlado como em contexto real de utilização, permitiram validar o funcionamento da aplicação e a sua adequação ao problema identificado. A execução em dispositivos físicos e a interação com utilizadores externos evidenciaram que a solução apresenta boa usabilidade e potencial de aplicação prática. Embora a aplicação cubra os 308 concelhos do país nas dimensões de escolas, hospitais e criminalidade, a cobertura dos dados de renda fica limitada aos 24 concelhos para os quais o INE publica preços medianos oficiais, conforme decisão metodológica da fonte. Para os restantes municípios, esta limitação é sinalizada de forma transparente na interface, sem recurso a imputação de valores.

Do ponto de vista técnico, o projeto implicou a superação de diversos desafios, nomeadamente na integração entre diferentes tecnologias, no tratamento de dados provenientes de múltiplas fontes e na otimização do desempenho na visualização geográfica. Estas dificuldades contribuíram para um processo de aprendizagem relevante, consolidando conhecimentos nas áreas de desenvolvimento mobile, sistemas distribuídos, bases de dados e processamento de dados.

Em termos de trabalho futuro, existem várias possibilidades de evolução da aplicação, incluindo a melhoria da qualidade e atualização dos dados, a implementação de funcionalidades adicionais como alertas personalizados, o reforço dos mecanismos de segurança e a expansão para outras regiões geográficas. Adicionalmente, a integração mais aprofundada de inteligência artificial poderá permitir fornecer recomendações mais personalizadas e contextualizadas ao utilizador.

Em síntese, o RentScope constitui uma solução consistente e funcional, com potencial de evolução, que demonstra a aplicação prática de conhecimentos adquiridos ao longo do curso e a capacidade de desenvolver um sistema completo orientado para a resolução de um problema real.

Bibliografia

Anthropic. (2026). Claude. Disponível em: <https://claude.ai>

Direção-Geral do Território. (2024). Carta Administrativa Oficial de Portugal (CAOP).

FastAPI. (2024). FastAPI Documentation. Disponível em: <https://fastapi.tiangolo.com>

Google. (2024). Google Maps SDK for Android. Disponível em:
<https://developers.google.com/maps/documentation/android-sdk>

Google. (2024). Jetpack Compose. Disponível em:
<https://developer.android.com/jetpack/compose>

Instituto Nacional de Estatística. (2024). Dados estatísticos sobre habitação e indicadores territoriais. Disponível em: <https://www.ine.pt>

JetBrains. (2024). Kotlin Programming Language. Disponível em: <https://kotlinlang.org>

Nielsen, J. (1994). Usability Engineering. Morgan Kaufmann.

Nielsen, J. (2000). Why you only need to test with 5 users. Nielsen Norman Group. Disponível em: <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>

OpenAI. (2025). ChatGPT. Disponível em: <https://chat.openai.com>

Render. (2024). Render Documentation. Disponível em: <https://render.com/docs>

Retrofit. (2024). Retrofit Library. Disponível em: <https://square.github.io/retrofit>

Supabase. (2024). Supabase Documentation. Disponível em: <https://supabase.com/docs>

Durante o desenvolvimento do projeto, foram utilizadas ferramentas de inteligência artificial, nomeadamente modelos de linguagem da OpenAI (ChatGPT) e Anthropic (Claude), como apoio na estruturação de código, esclarecimento de conceitos e melhoria da redação técnica.

Anexo A — Guião de Testes e Resultados

Os testes com utilizadores foram realizados de forma iterativa, permitindo a melhoria progressiva da aplicação ao longo do processo de validação. Seguiu-se uma abordagem de iterative usability testing (Nielsen, 1994), na qual cada sessão de teste é seguida de um ciclo de correção antes da realização da sessão seguinte. Este processo foi repetido com os restantes utilizadores, permitindo refinar gradualmente a solução e corrigir limitações identificadas em fases anteriores. Nielsen (2000) demonstra que um conjunto reduzido de utilizadores é suficiente para identificar a maioria dos problemas de usabilidade significativos, o que valida a dimensão da amostra utilizada neste contexto. Esta abordagem possibilitou uma validação contínua da aplicação, garantindo que os principais problemas de usabilidade fossem resolvidos de forma progressiva, contribuindo para uma experiência de utilização mais consistente e intuitiva.

Tarefas e objetivos:

ID	Tarefa	Objetivo
T1	Abrir a aplicação	Validar acesso inicial
T2	Criar conta ou fazer login	Validar autenticação
T3	Selecionar continente e país (ex: Portugal)	Validar navegação
T4	Explorar o mapa	Validar compreensão visual
T5	Aplicar filtros	Validar interação com dados
T6	Identificar um município interessante	Validar interpretação
T7	Comparar duas localidades	Validar funcionalidade de comparação
T8	Guardar um município como favorito	Validar persistência
T9	Consultar histórico de preços	Validar análise temporal
T10	Fazer uma pergunta ao assistente IA	Validar funcionalidade inteligente

Tabela 19 - Tarefas e Objetivos dos Testes

Resultados:

Utilizador	Tarefa	Tarefas concluídas autonomamente	Resultado	Tempo	Observações
U1	T1–T10	5/10	O utilizador apresentou dificuldades generalizadas na navegação, não conseguindo concluir várias tarefas de forma autónoma.	00:15	Rever página de login e fluxo de verificação de e-mail; tornar municípios clicáveis no mapa; adicionar lógica de favoritos aos resultados; ajustar menu de navegação.
U2	T1–T10	8/10	O utilizador concluiu a maioria das tarefas com menor dependência de apoio, registando apenas dificuldades pontuais de navegação.	00:05	Melhorar a navegação entre ecrãs.
U3	T1–T10	9/10	O utilizador conseguiu utilizar a aplicação de forma autónoma e sem dificuldades significativas.	00:10	Adicionar ícone de visibilidade da palavra-passe no campo de login.

Tabela 20 - Resultados dos Testes

A evolução entre sessões — de 5/10 tarefas concluídas autonomamente (U1) para 9/10 (U3) — reflete a eficácia do ciclo iterativo adotado. Os problemas identificados em cada sessão foram corrigidos antes da seguinte, resultando numa experiência progressivamente mais fluida.

Questionário:

A aplicação foi fácil de utilizar?

U1: Não muito

U2: Em sua maior parte sim, alguns recursos não foram encontrados sem a ajuda do mediador

U3: Sim

O mapa ajudou a compreender os dados?

U1: Não

U2: Sim, apesar de dúvidas sobre os indicadores

U3: Sim

A funcionalidade de IA foi útil?

U1: Sim

U2: Sim

U3: Sim

A resposta negativa de U1 relativamente ao mapa ocorreu antes da implementação das correções de interatividade, nomeadamente a clicabilidade dos municípios. As sessões seguintes confirmaram que os ajustes realizados resolveram essa limitação.